PACKET CAPTURE: OVERVIEW AND BASICS

Byeong-hee Roh
Dept. of Software and Computer Engineering
Ajou University

변화와 융합의 중심
MMcN
멀티미디어 융합 연구실
Mobile Multimedia convergecne Network Lab.
http://mmcn.ajou.ac.kr

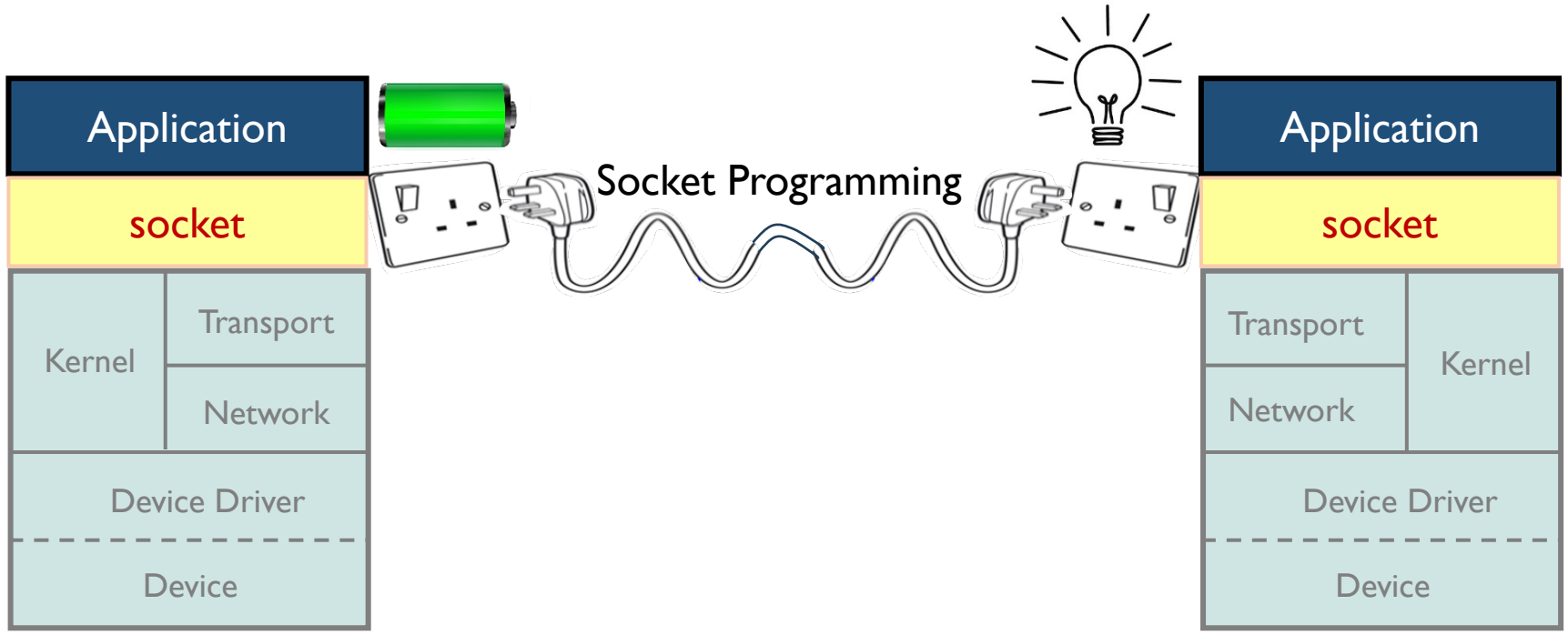
Contents

- Sockets
- Packet Capture Basics
- Understanding Libpcap and Npcap
- Pcap Data Structure
- Example – Captured File Analysis

SOCKETS

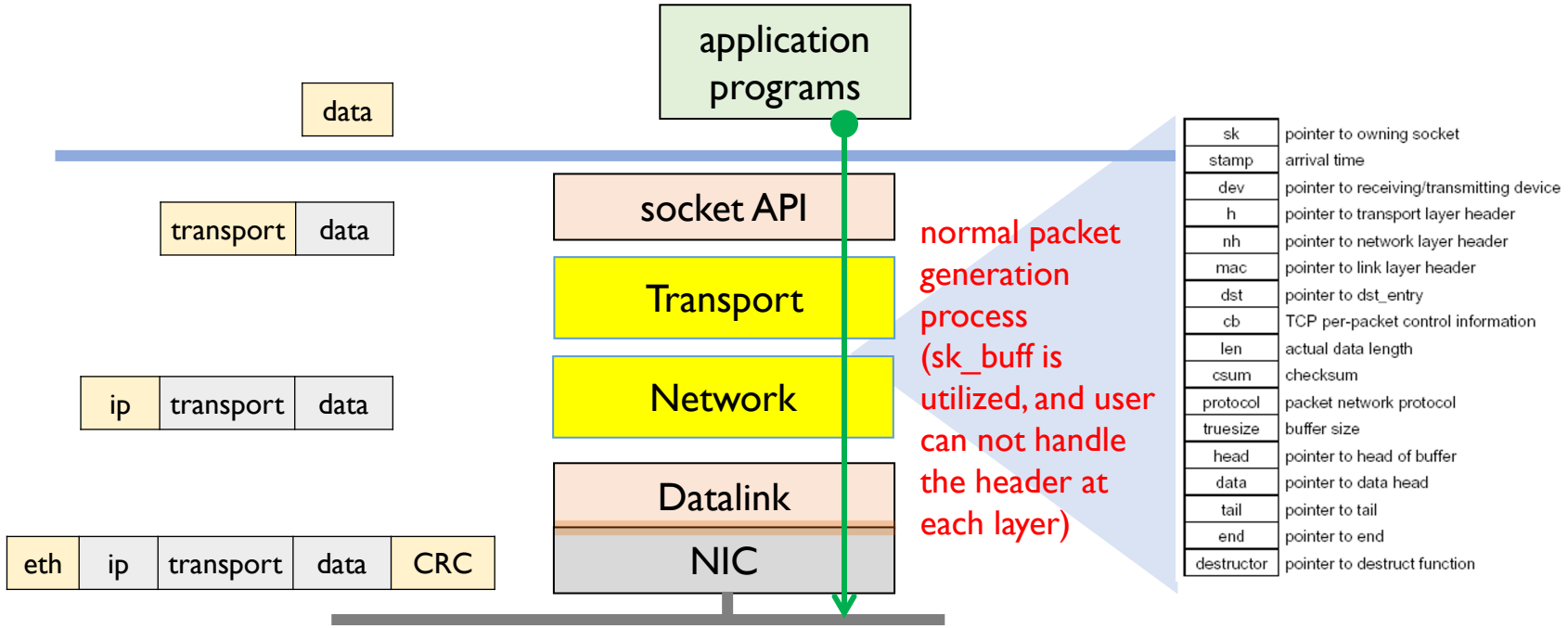
Network Application Programming

- Sockets provide
  - network programming interface between
    - processes at application layers on the network
    - through TCP/IP network kernel (OS), but don't care of it



Standard Socket (1/3)

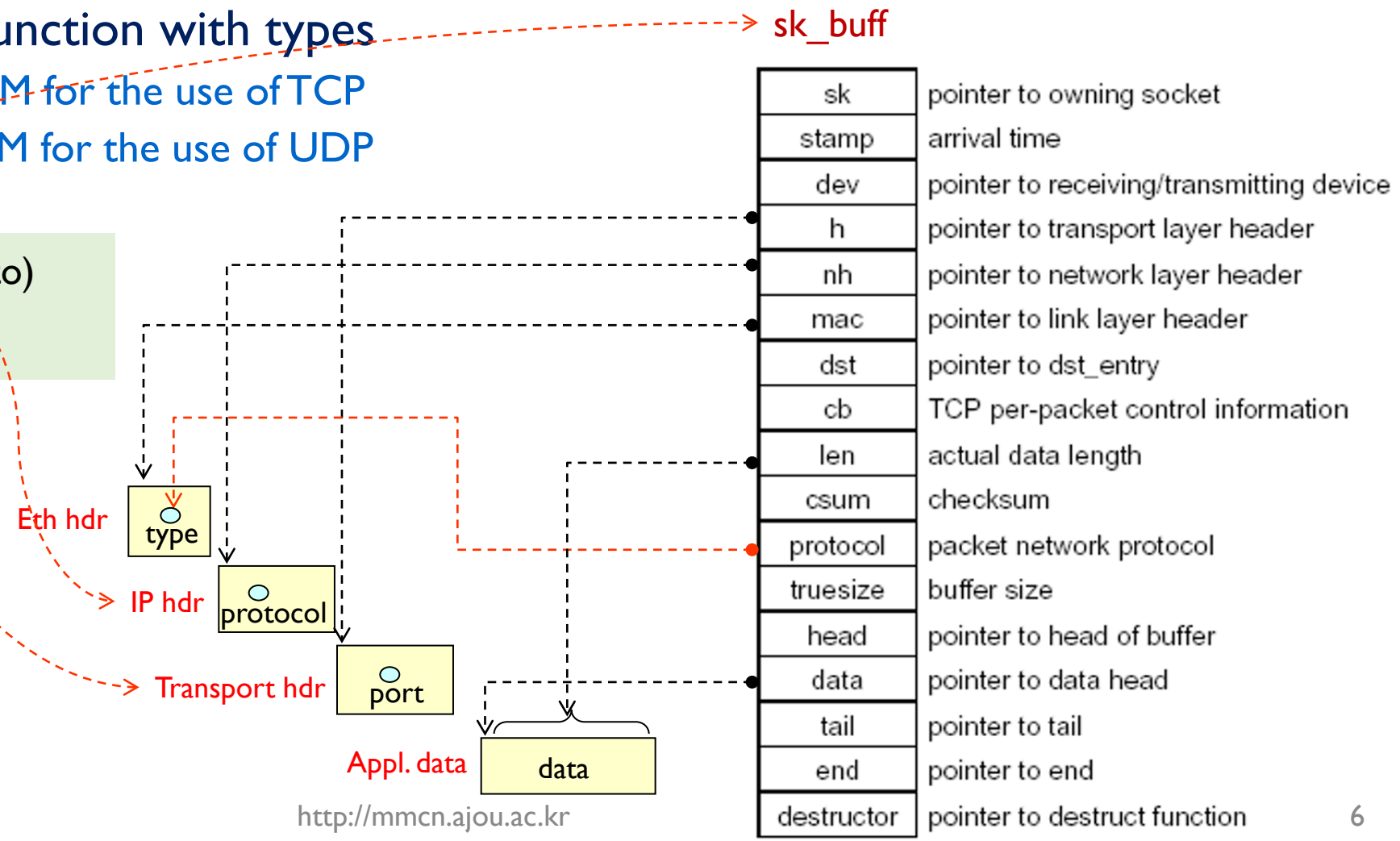
- In standard sockets,
  - the SDU from upper layer is encapsulated at a layer
  - TCP/UDP, IP, and Frame headers are automatically added, and
  - user don't need to aware the existence of TCP/IP layer



Standard Socket (2/3)

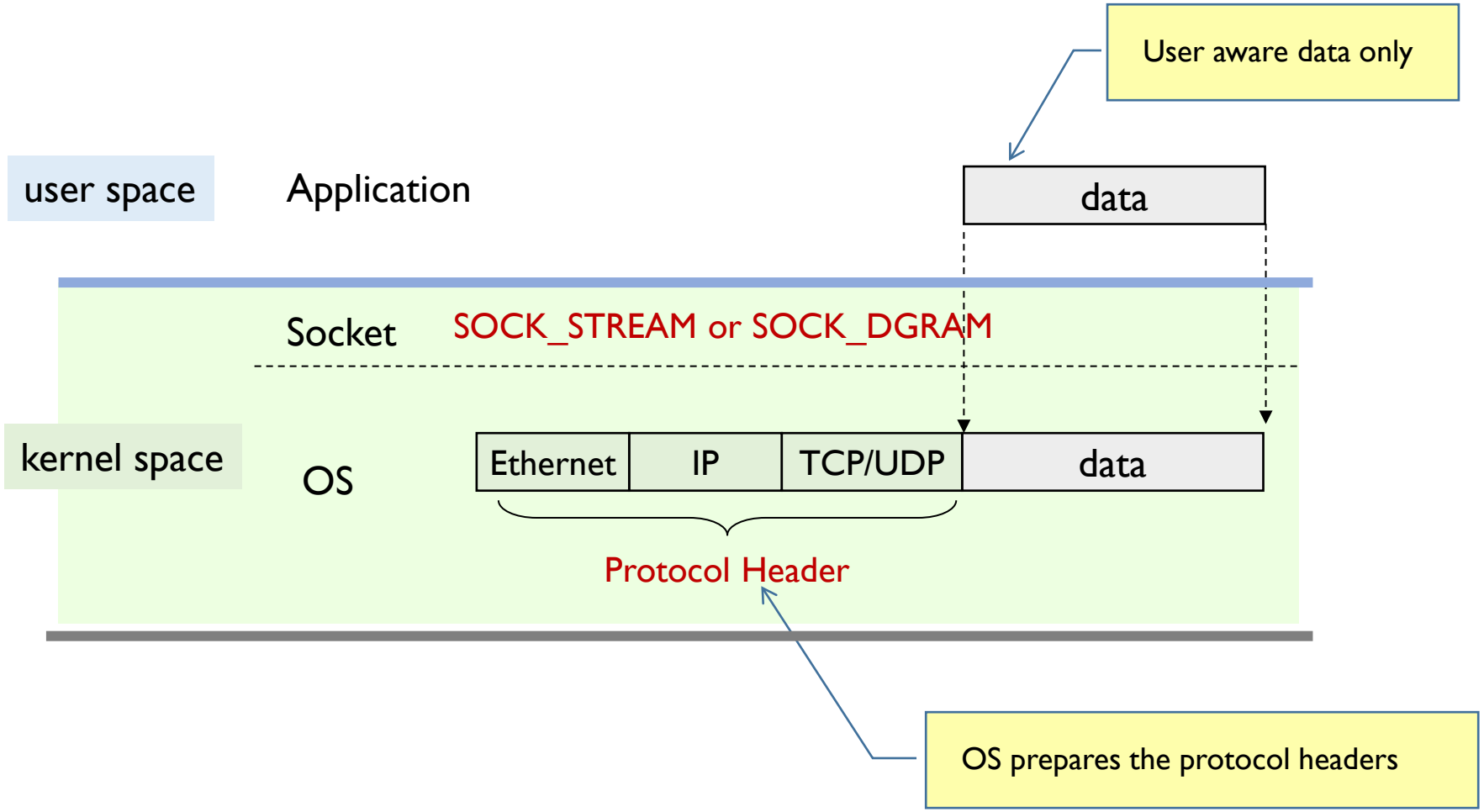
- To create standard sockets
  - use socket( ) function with types
    - SOCK_STREAM for the use of TCP
    - SOCK_DGRAM for the use of UDP

```
sd = socket (dm, type, proto)
z = bind (sd, addr, len)
```



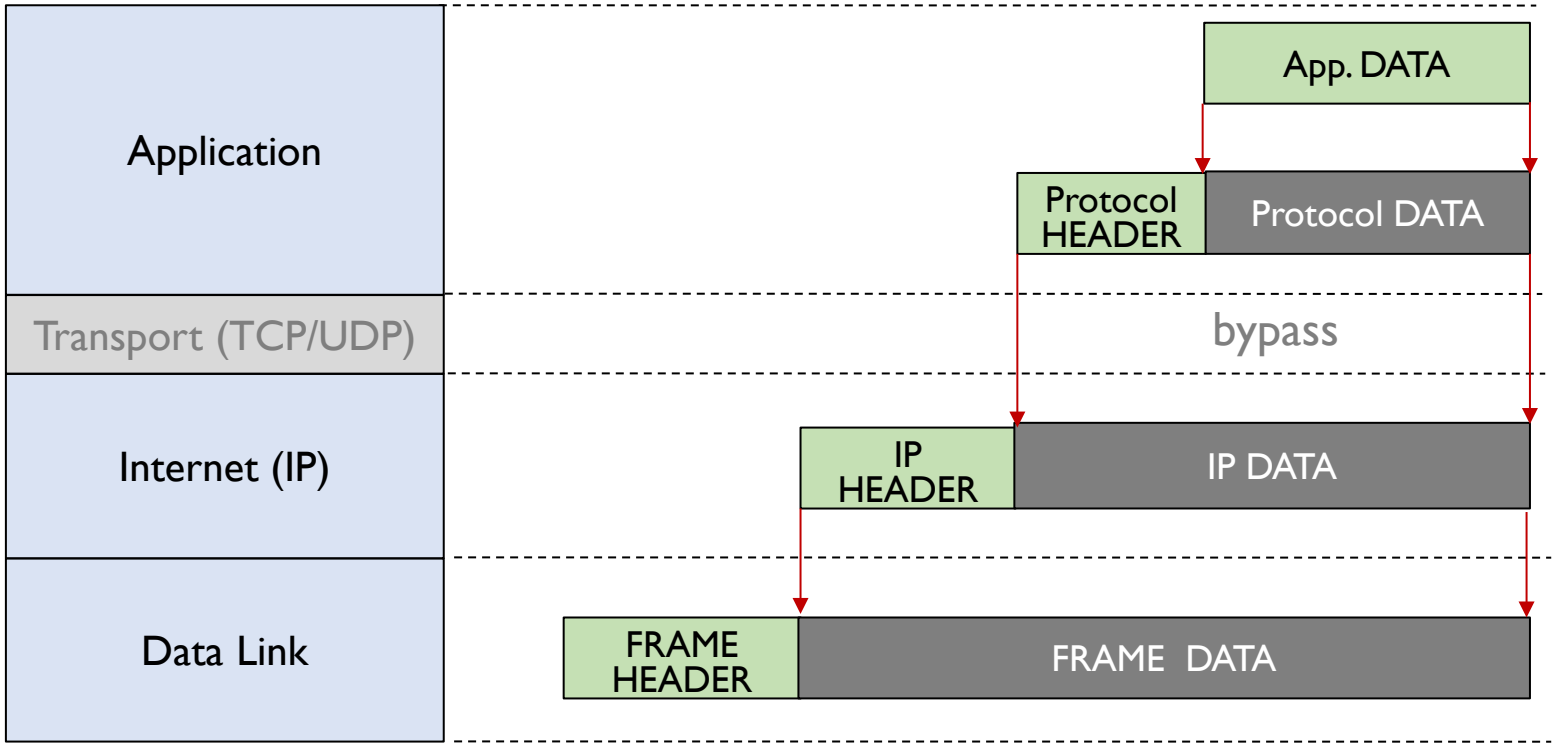
Standard Socket (3/3)

❖ SOCK_STREAM and SOCK_DGRAM



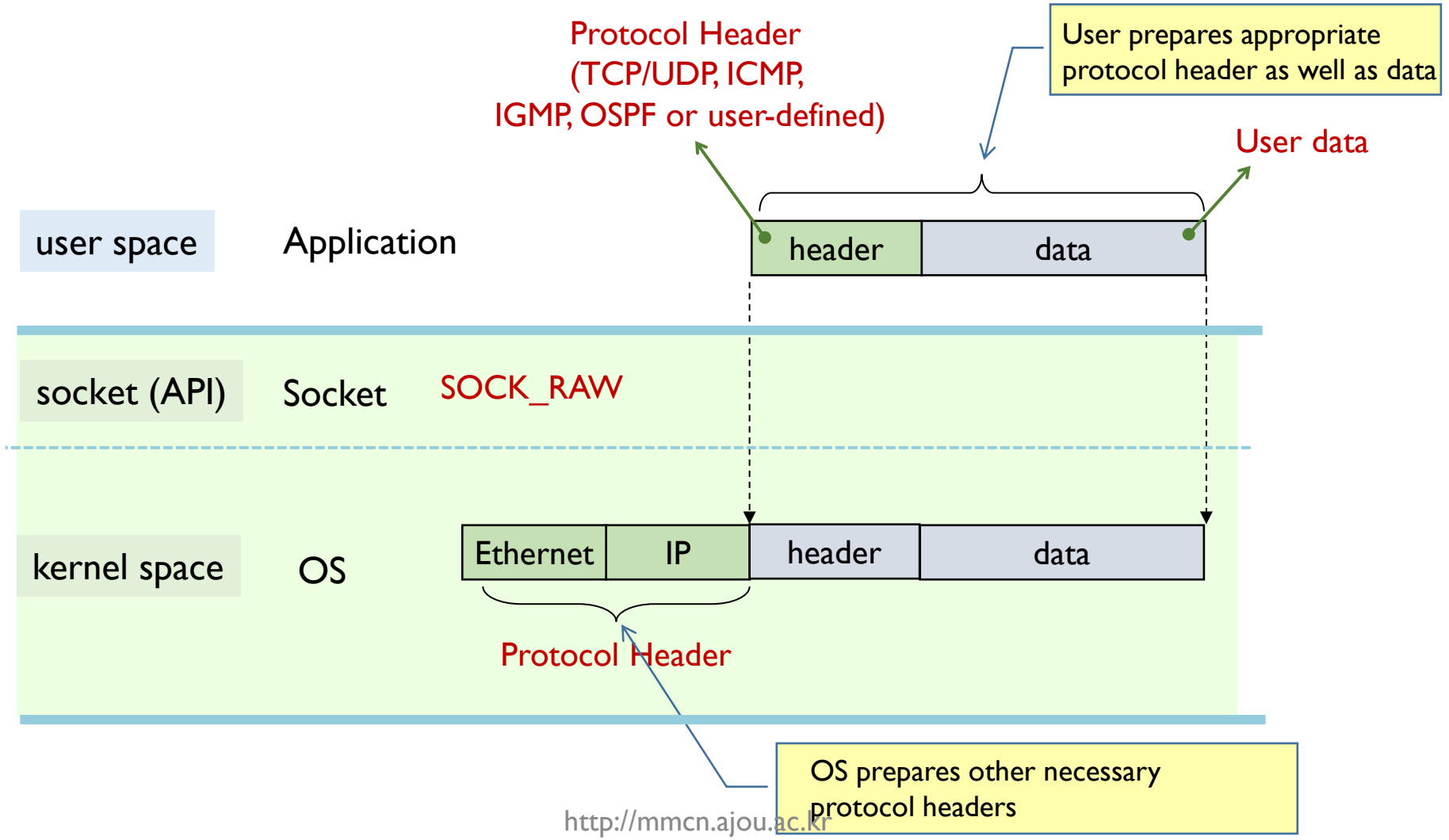
Raw Socket (1/2)

- A raw socket is
  - a type of socket that allows bypass the transport layer
  - user should prepare IP data part with appropriate headers and data



Raw Socket (2/2)

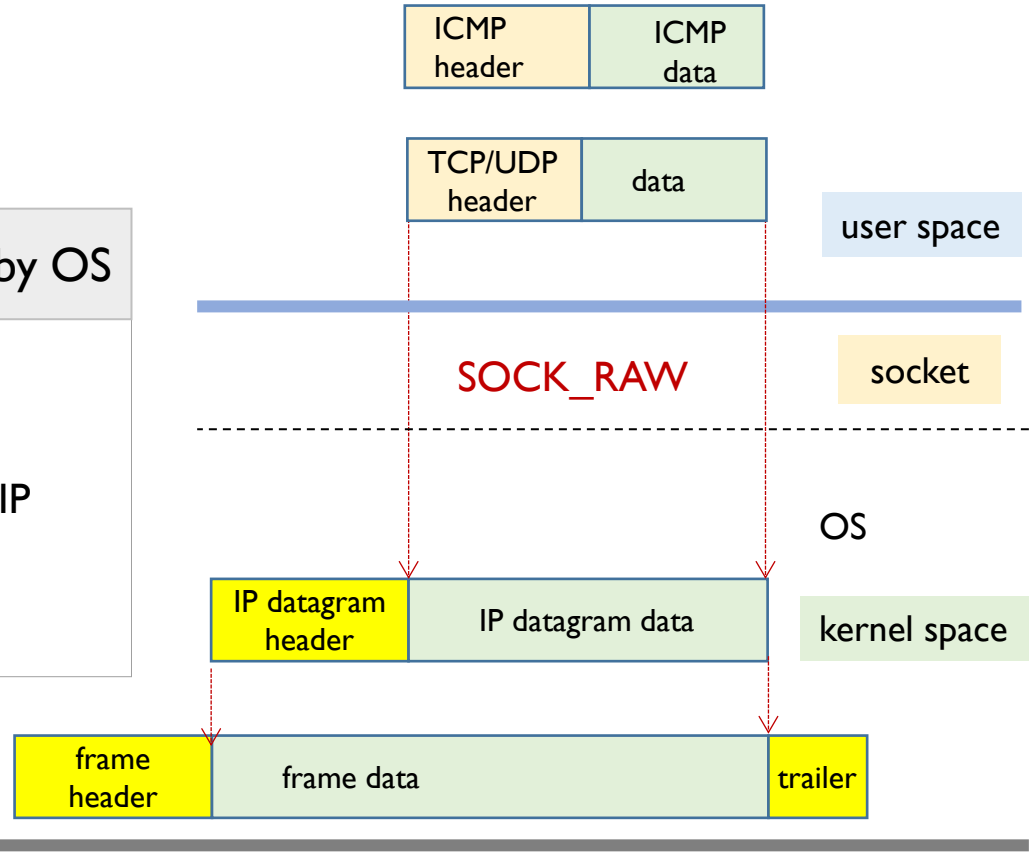
❖ RAW Socket



RAW Socket with IP_HDRINCL option (1/3)

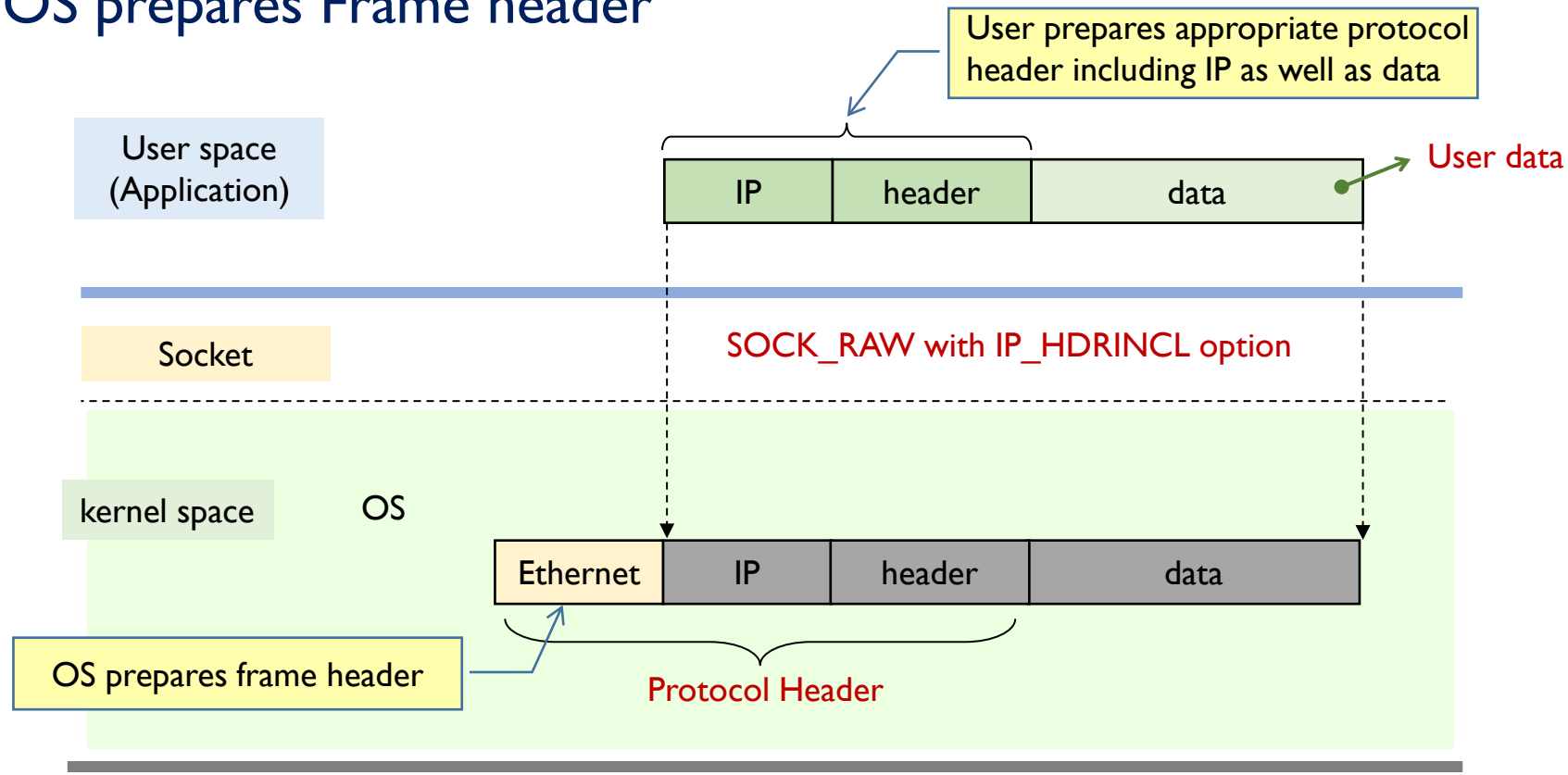
- SOCK_RAW without IP_HDRINCL option
  - it bypasses Transport layer only
  - OS prepares IP header at network layer

Protocol	Header given by Application	Header by OS
IPPROTO_ICMP	ICMP (ICMP Header + ICMP Data)	Eth+IP
IPPROTO_IGMP	IGMP (IGMP Header + IGMP Data)	
IPPROTO_TCP	TCP (TCP Header + App. Data)	
IPPROTO_UDP	UDP (UDP Header + App. Data)	
user defined	user defined header (IP + USER)	



RAW Socket with IP_HDRINCL option (2/3)

- SOCK_RAW with IP_HDRINCL option
  - it bypasses both transport and IP layer
  - OS prepares Frame header



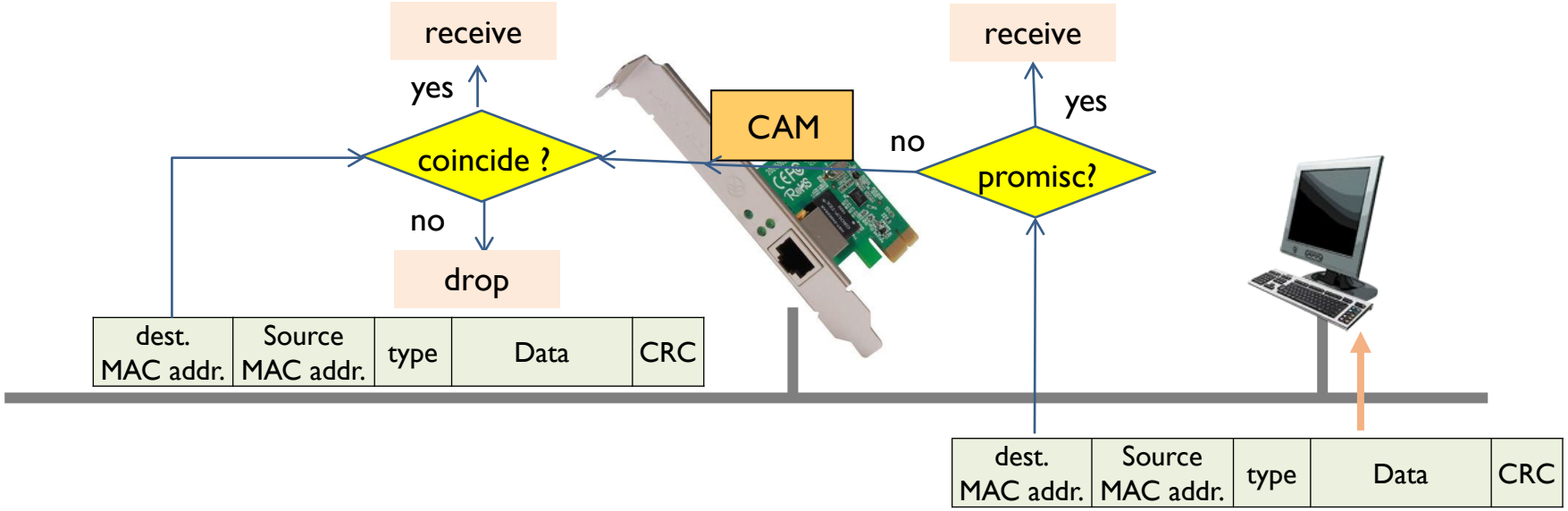
RAW Socket with IP_HDRINCL option (3/3)

- Limitations of IP_HDRINCL opetion
  - IP_HDRINCL only allows for creating IP headers on outgoing packets
    - it does not provide any functionality for capturing or analyzing incoming traffic
    - so, it can't be used for packet analysis
  - IP_HDRINCL requires IP header by the application for each packet sent
    - it does not inherently handle lower layers, such as the Ethernet layer
  - IP_HDRINCL depends on system-specific behavior
    - Some Oss may not fully support or implement this option in a consistent way
    - Windows does not support TCP packet transmission with IP_HDRINCL option
- libpcap
  - preferable and comprehensive packet capture and analysis library
    - compared to IP_HDRINCL, which is more specialized and limited to sending custom IP packets only

PACKET CAPTURE BASICS

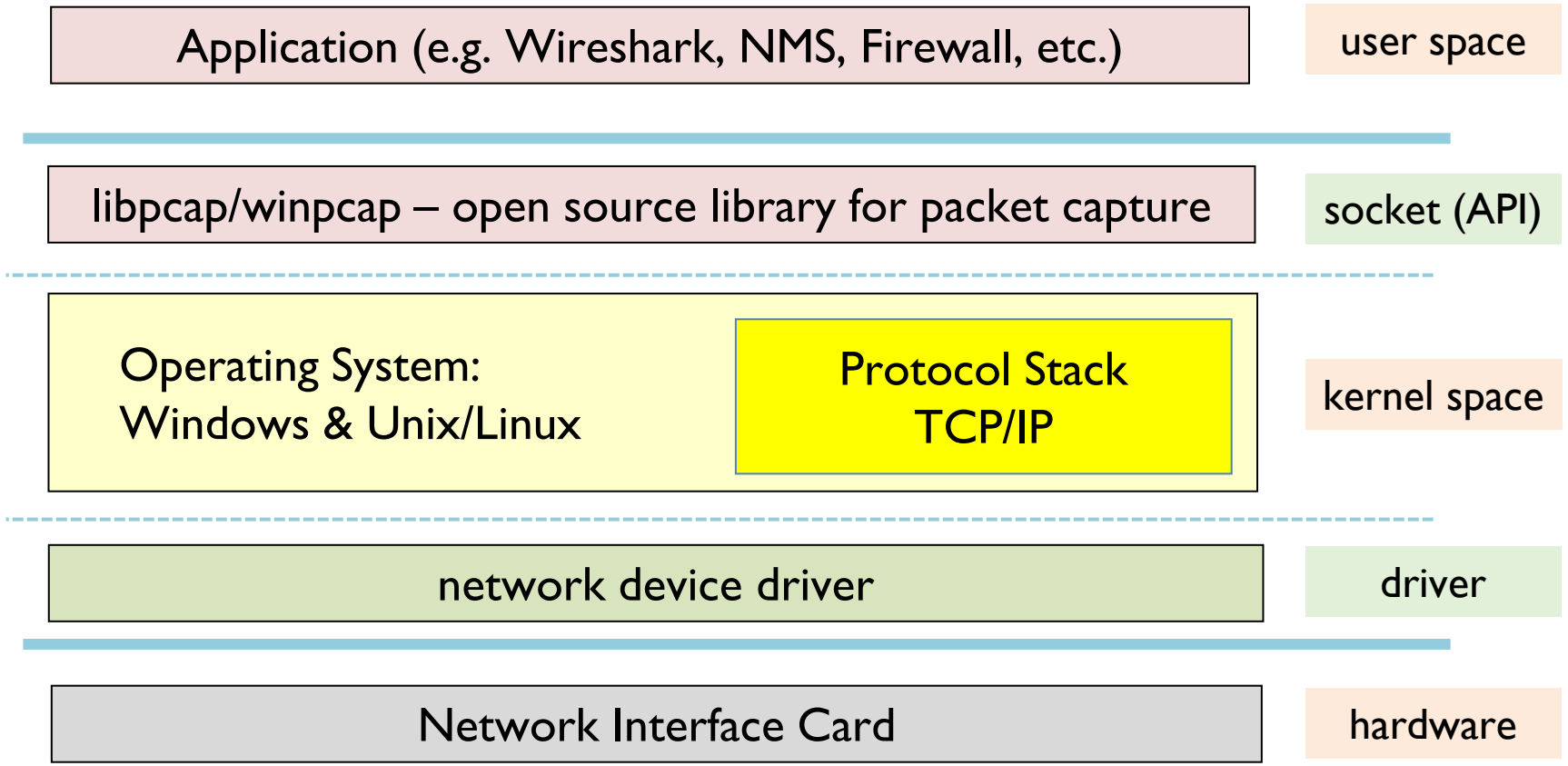
Network Interface Card (NIC)

- NIC accepts frames with their destination addresses
  - its MAC address – stored in EEPROM
  - multicast addresses – stored in Content Addressable Memory(CAM), and
  - broadcast address
- Promiscuous Mode
  - NIC accepts all frames it receives



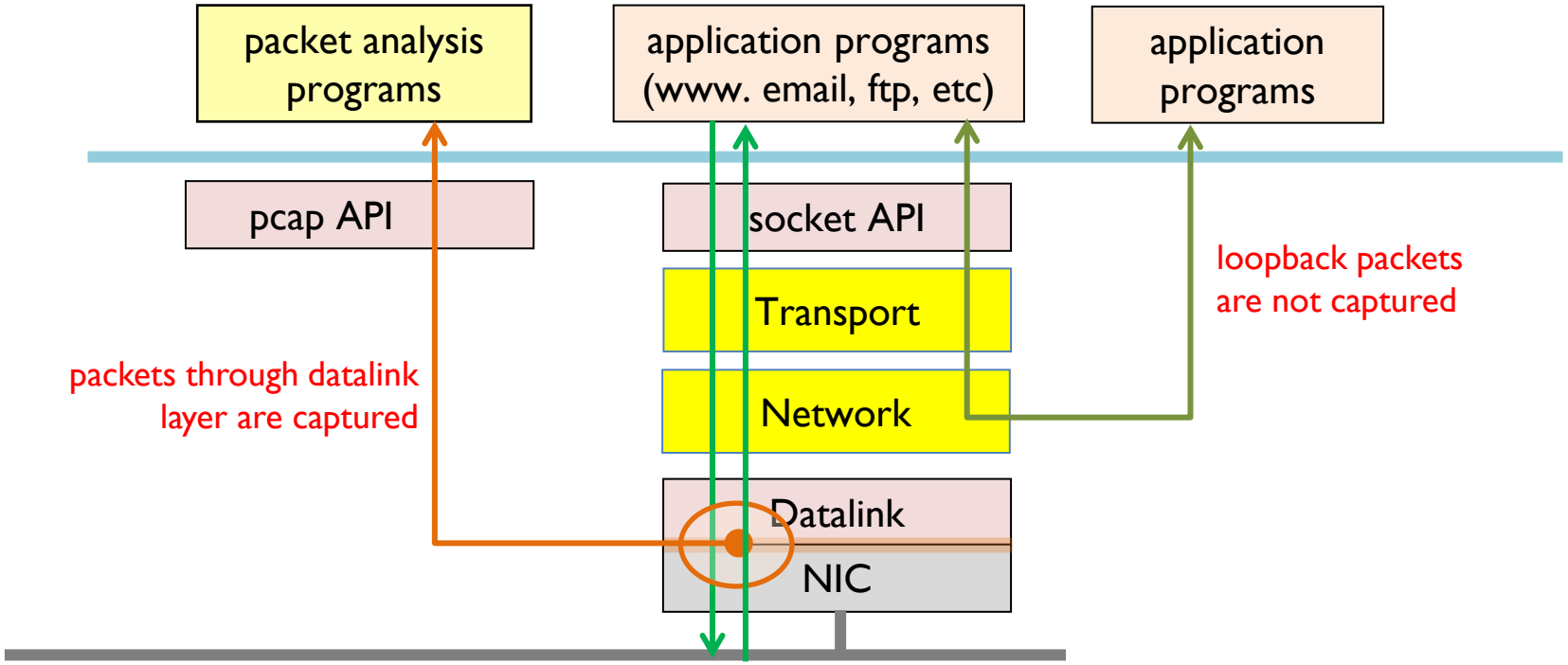
Packet Capture Structure

❖ Wireshark, OS, and Pcap library



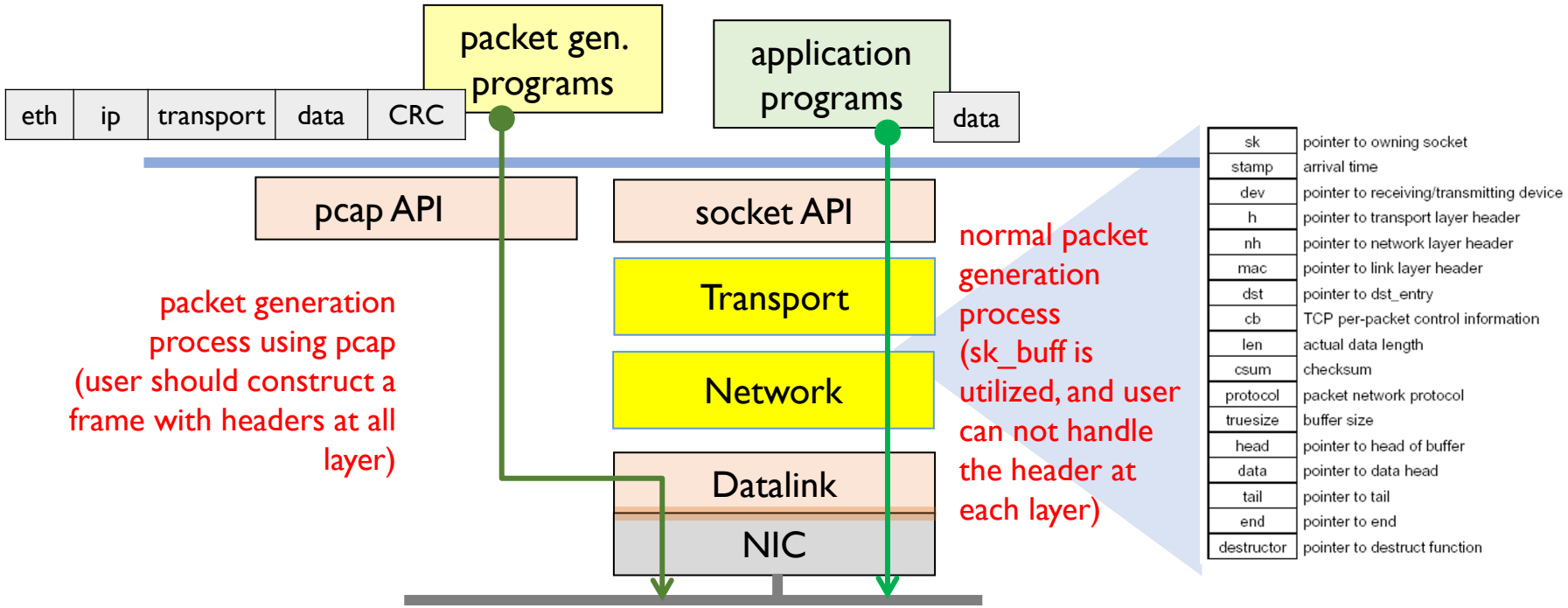
Packet Capture Structure

- Packet Capture Layer
  - Packets are not captured at all protocol layers,
  - Packets in and out through NIC (data link layer) are captured.
    - with pcap library, frames received and transmitted at datalink layer are delivered to application programs by passing tcp/ip stack



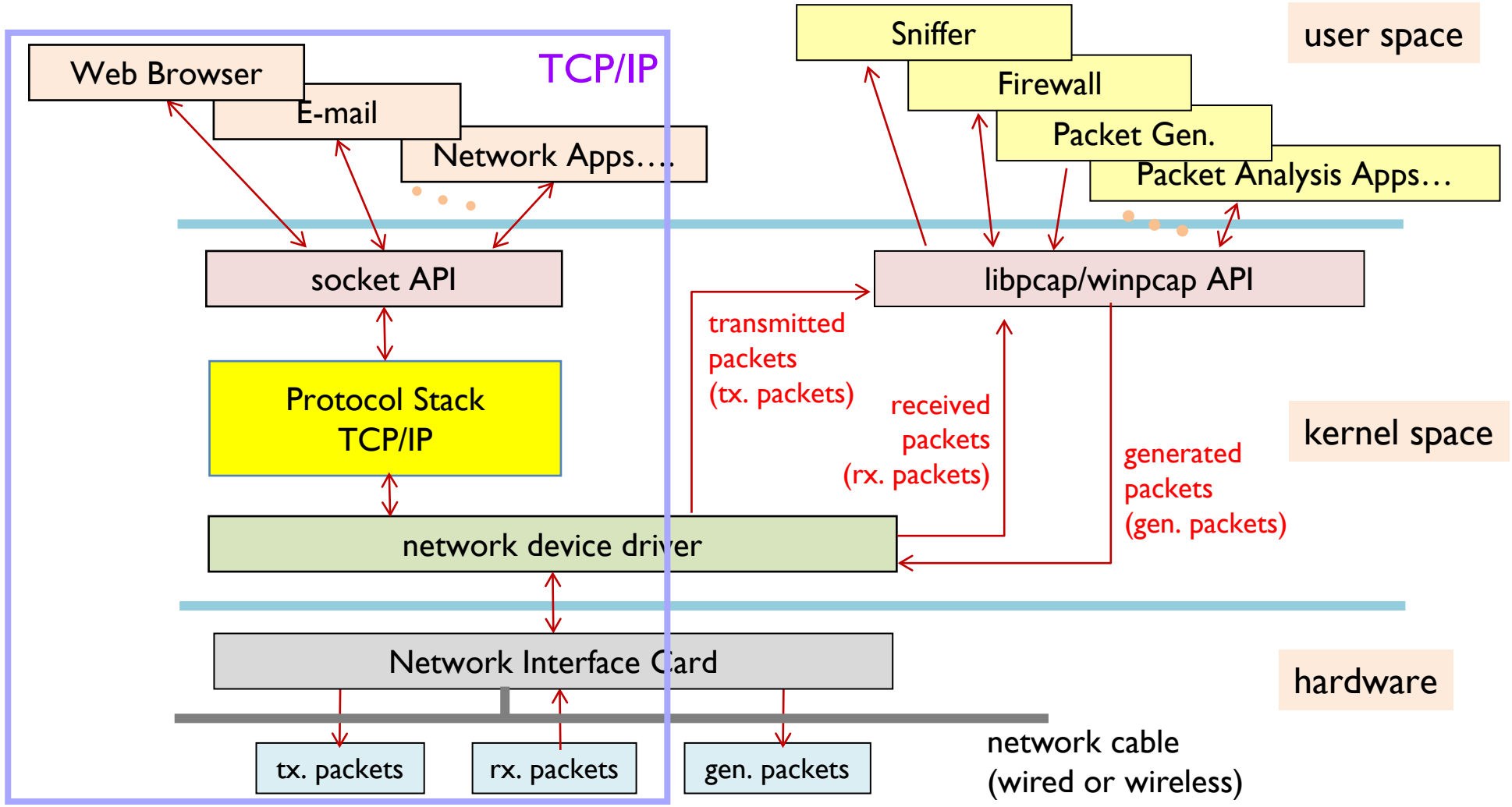
Packet Generation Structure

- pcap API provides packet generation process
  - an application program
    - constructs a full frame format with headers at all layers including application data, then
    - sends it through NIC directly using pcap library



Packet Analysis Process

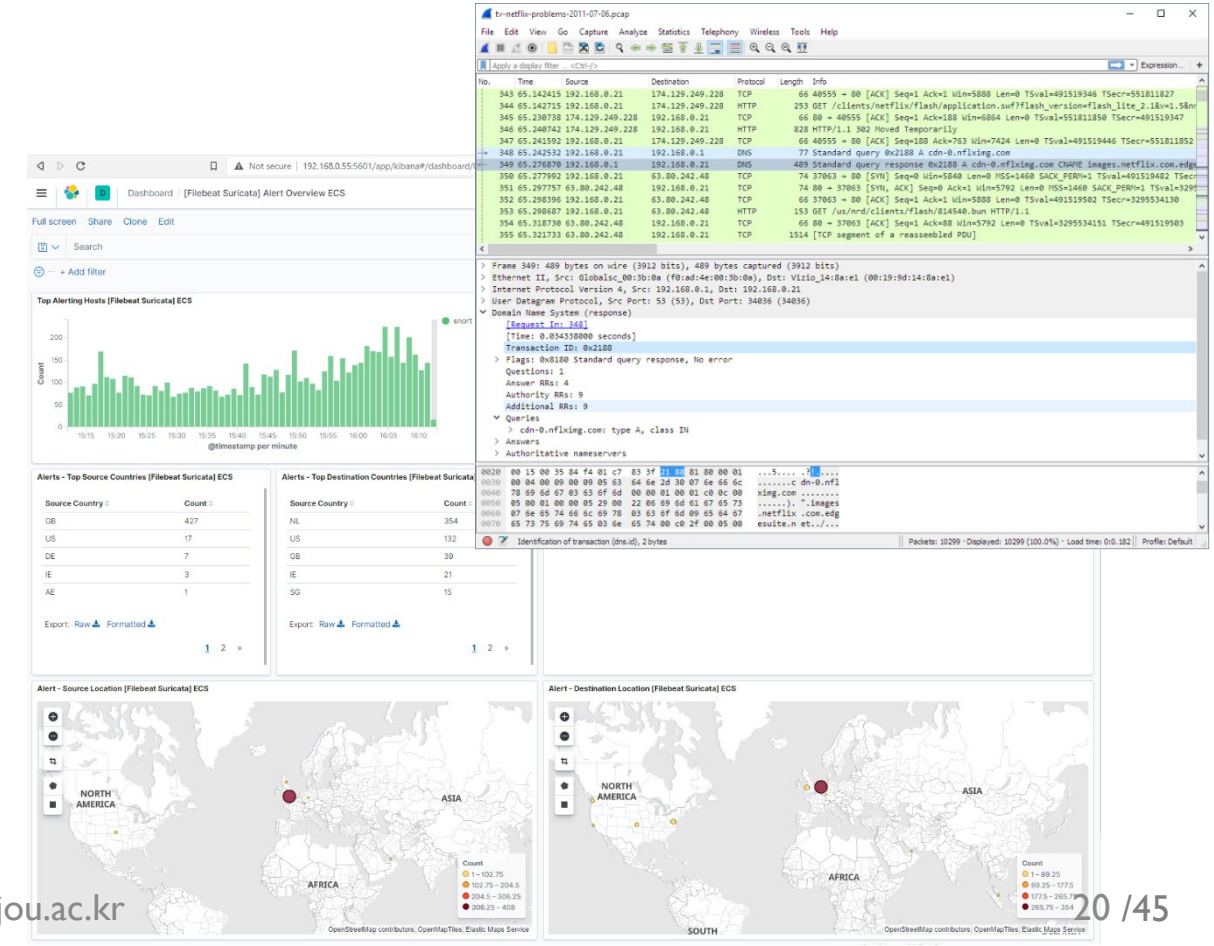
- Packet sending, capture, and generation



Understanding Libpcap and Npcap

pcap library

- pcap (packet capture)
  - open source library providing application programming interface (API) for packet-capture and network analysis
  - written in C/C++ basically
  - capable of
    - to capture network packets
    - to save captured packets
    - to analyze the real-time and saved packets
    - to inject packets, and so on



libpcap on Linux (1/2)

- libpcap on Linux
  - developed in 1994
    - by the tcpdump developers in the Network Research Group at Lawrence Berkeley Laboratory
    - http://www.tcpdump.org
    - https://github.com/the-tcpdump-group
  - installed by default on the BSDs, macOS, and Ubuntu Linux
    - Latest Releases
      - tcpdump: version: 4.99.5, released on Aug. 2024
      - libpcap: version 1.10.5, released on Aug. 2024
    - Checking the installation of libpcap

```
tcpdump --version
```

    - ex)

```
bhroh@mmcn:~$ tcpdump --version
tcpdump version 4.99.4
libpcap version 1.10.4 (with TPACKET_V3)
OpenSSL 3.0.13 30 Jan 2024
```

libpcap on Linux (2/2)

- libpcap development package: libpcap-dev
  - includes the header files and development tools
    - needed to use the libpcap library in C or C++ programs
  - allows programs to access libpcap functionality
    - by providing files such as pcap.h
    - note: libpcap itself is a packet capture library used for capturing and analyzing network traffic. If the libpcap-dev package is not installed, pcap.h will be missing, leading to compilation errors when libpcap functions are used.
  - install the libpcap-dev

```
sudo apt update
sudo apt install libpcap-dev
```

Npcap on Windows

- WinPcap
  - the Windows version of libpcap
    - https://www.winpcap.org/
  - still available, but has not been maintained or updated since 2013
- Npcap
  - an architecture for packet capture and network analysis for Windows OS
    - all of WinPcap's packet capture and injection features are included,
    - with a few great additions like raw 802.11 frame capture
  - Npcap facilities
    - capture raw packets
    - filter the packets according to user-specified rules
    - transmit raw packets to the network
    - gather statistical information on the network traffic engine
    - support for remote packet capture.

Npcap on Windows

❖ Comparisons of features for Npcap and WinPcap

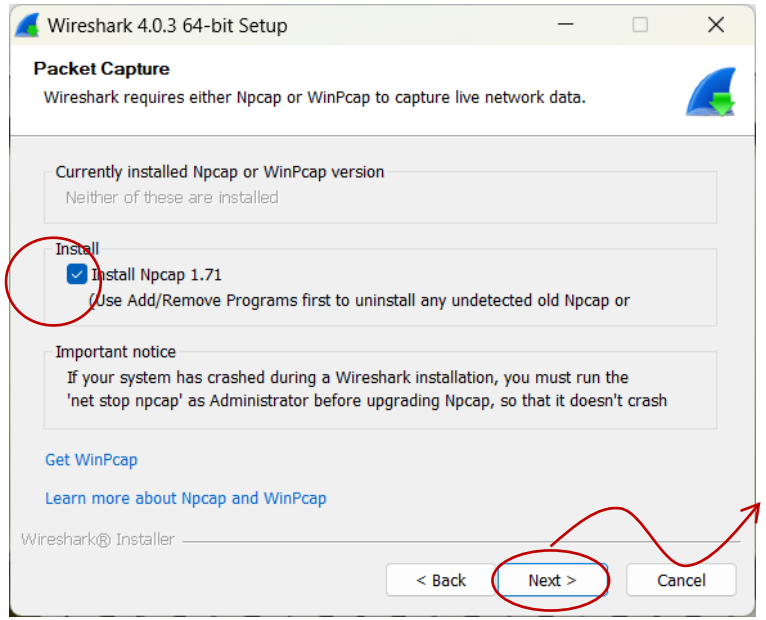
	Feature	Npcap	WinPcap
information	Actively maintained and supported	Yes	No (terminated)
	Latest release	Sep. 16, 2024	Mar. 8, 2013
	libpcap version	1.10.4 (2023)	1.0.0 (2008)
	License	Free for personal use	BSD-style
	Supported commercial/redistributable version	Yes (Npcap OEM)	No (terminated)
security	EV SHA-256 code signing	Yes	No
	Limit access to administrators (optional)	Yes	No
Basic Features	Packet capture with the cross-platform libpcap API	Yes	Yes
	Link-layer packet injection	Yes	Yes
	Source code available	Yes	Yes
Advanced Features	Capture raw 802.11 frames	Yes (with many adapters)	Yes (with specialized AirPcap)
	Capture Loopback traffic	Yes	No
	Inject Loopback traffic	Yes	No

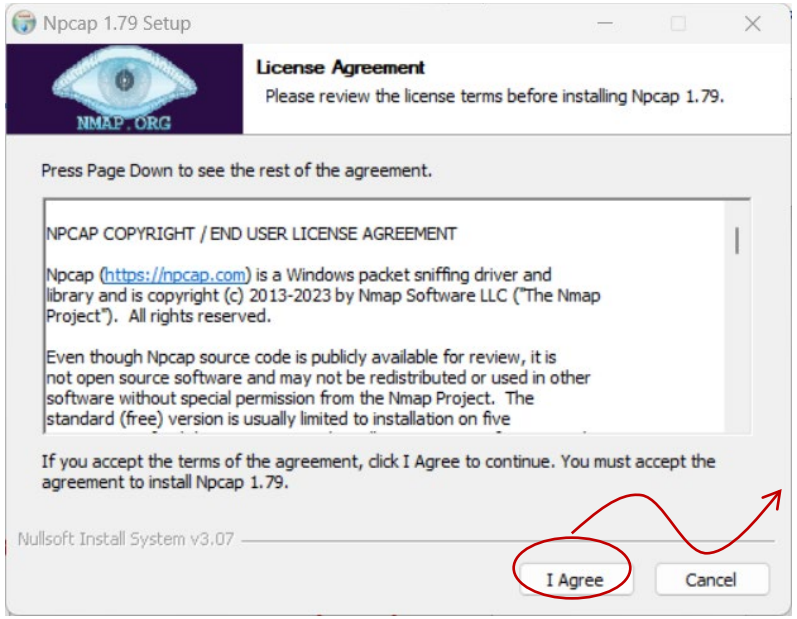
Source:https://nmap.org/npcap/vs-winpcap.html

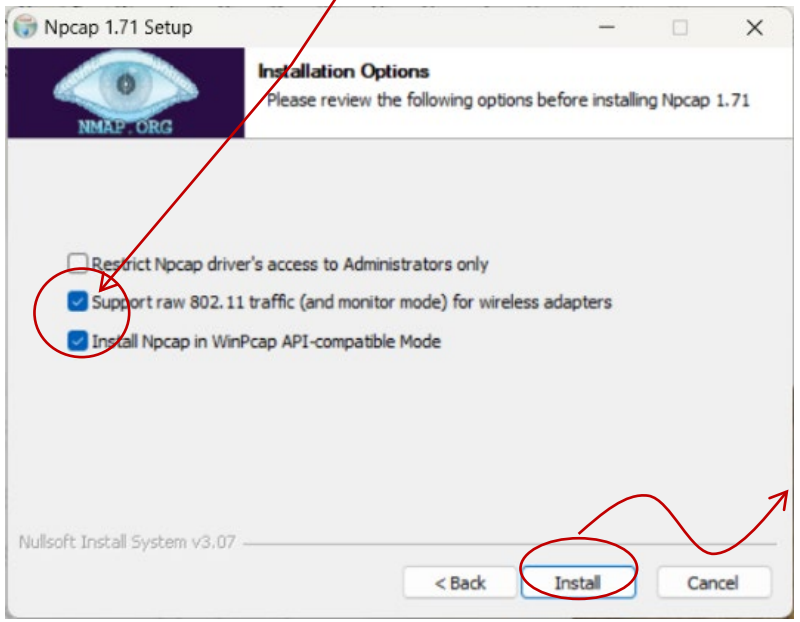
Npcap on Windows

- install Npcap
  - install during Wireshark installation
  - download Npcap installer at https://npcap.com/#download
    - Npcap 1.80 installer for Windows 7, 8, 8.1, 10, 11 (x86, x64 and ARM64)
    - Latest development source: Github (https://github.com/nmap/npcap)

only for 802.11 WiFi packet capture, but the wifi adapator should support monitor mode. Find adaptors in monitor mode at https://secwiki.org/w/Npcap/WiFi_adapters



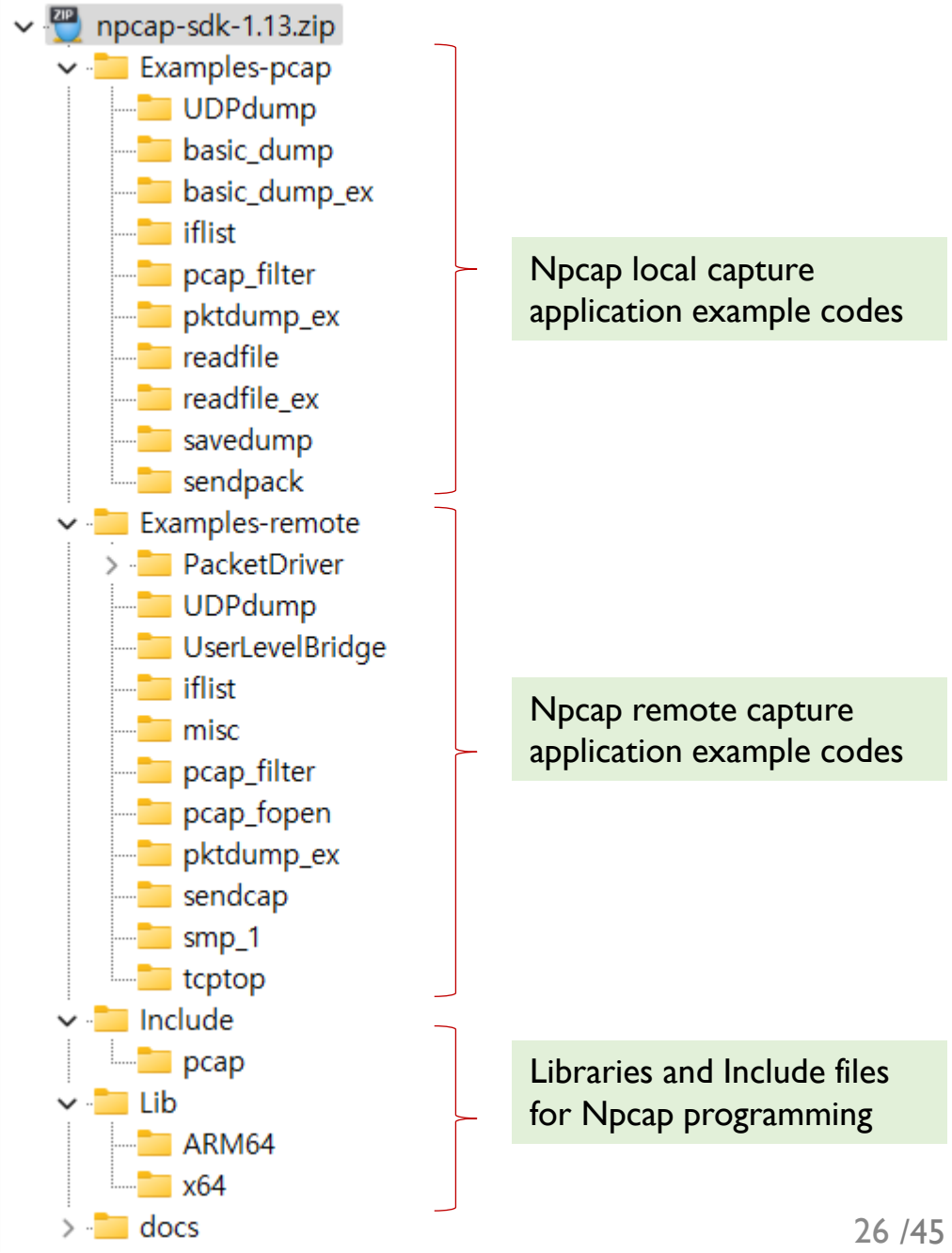




during wireshark install

Npcap Programming

- Npcap SDK
  - download Npcap SDK
    - at https://npcap.com/#download
    - Npcap SDK 1.13 (ZIP)
  - unpack the SDK
- Resources
  - Development Tutorial
    - https://nmap.org/npcap/guide/npcap-tutorial.html
  - User's Guide
    - https://nmap.org/npcap/guide/
  - WiFi adaptors in monitor mode
    - https://secwiki.org/w/Npcap/WiFi_adapters
  - Comparisons to WinPcap
    - https://nmap.org/npcap/vs-winpcap.html

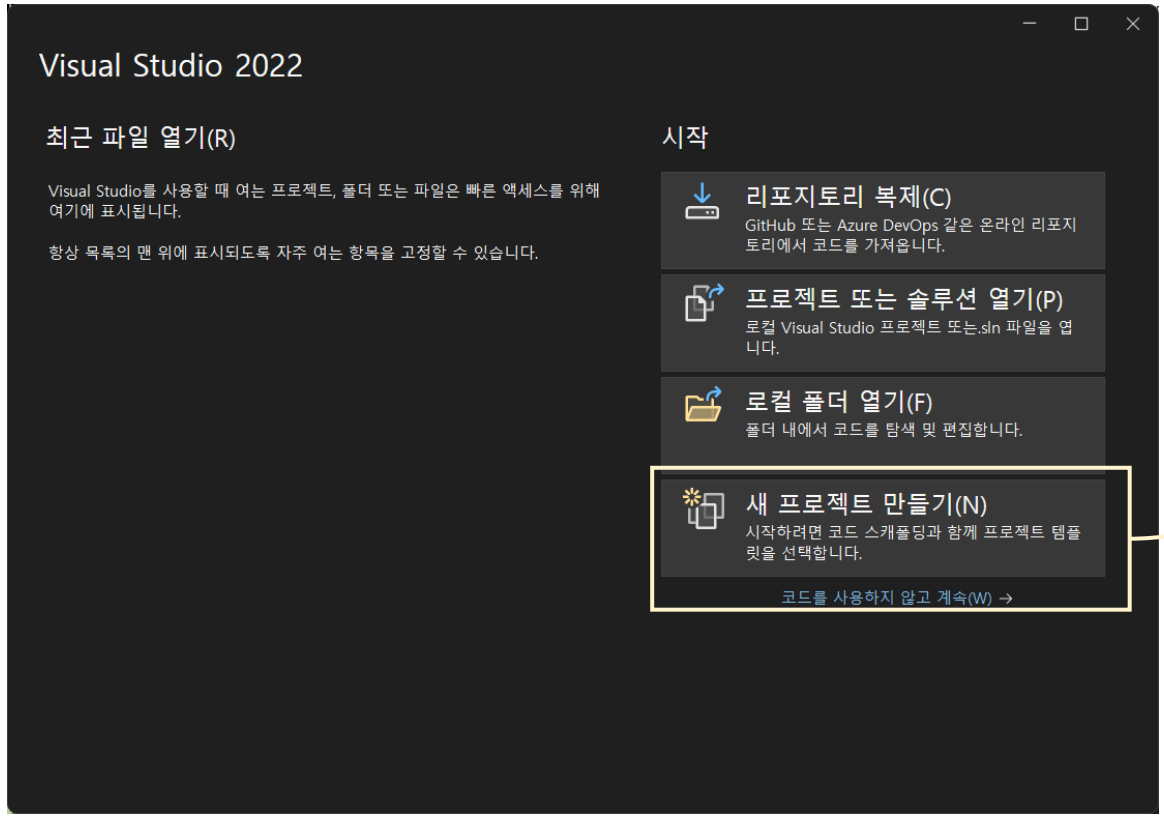


Npcap Programming with Visual Studio

- Configuring MS Visual Studio compilation environments
  - Include the file <pcap.h> at the beginning of every source file
    - that uses the functions exported by library.
  - Add directories for include and library files, for example
    - NpcapSDK\include for include directory
    - NpcapSDK \lib for library directory
  - Include the following preprocessor definitions
    - WPCAP for Win32 specific functions of Npcap
    - HAVE_REMOTE for the remote capture capabilities of Npcap
  - Set the options of the linker to include the wpcap.lib
  - Set the options of the linker to include the winsock library file ws2_32.lib.

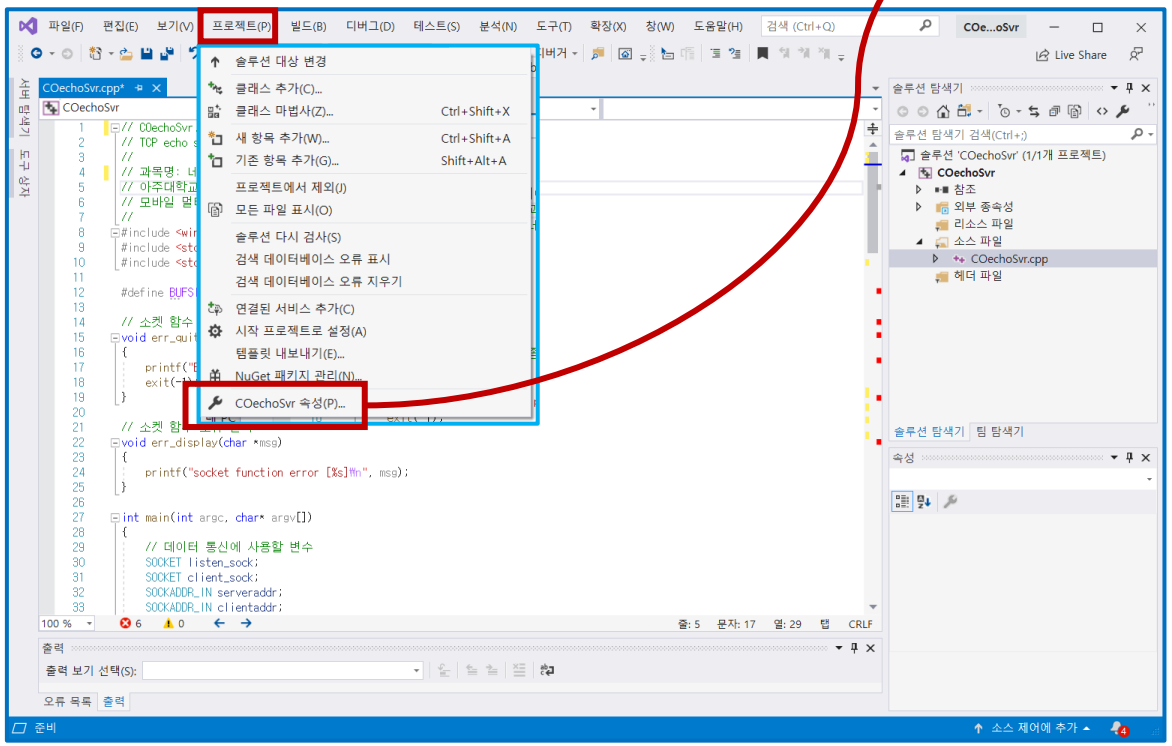
Npcap Programming with Visual Studio

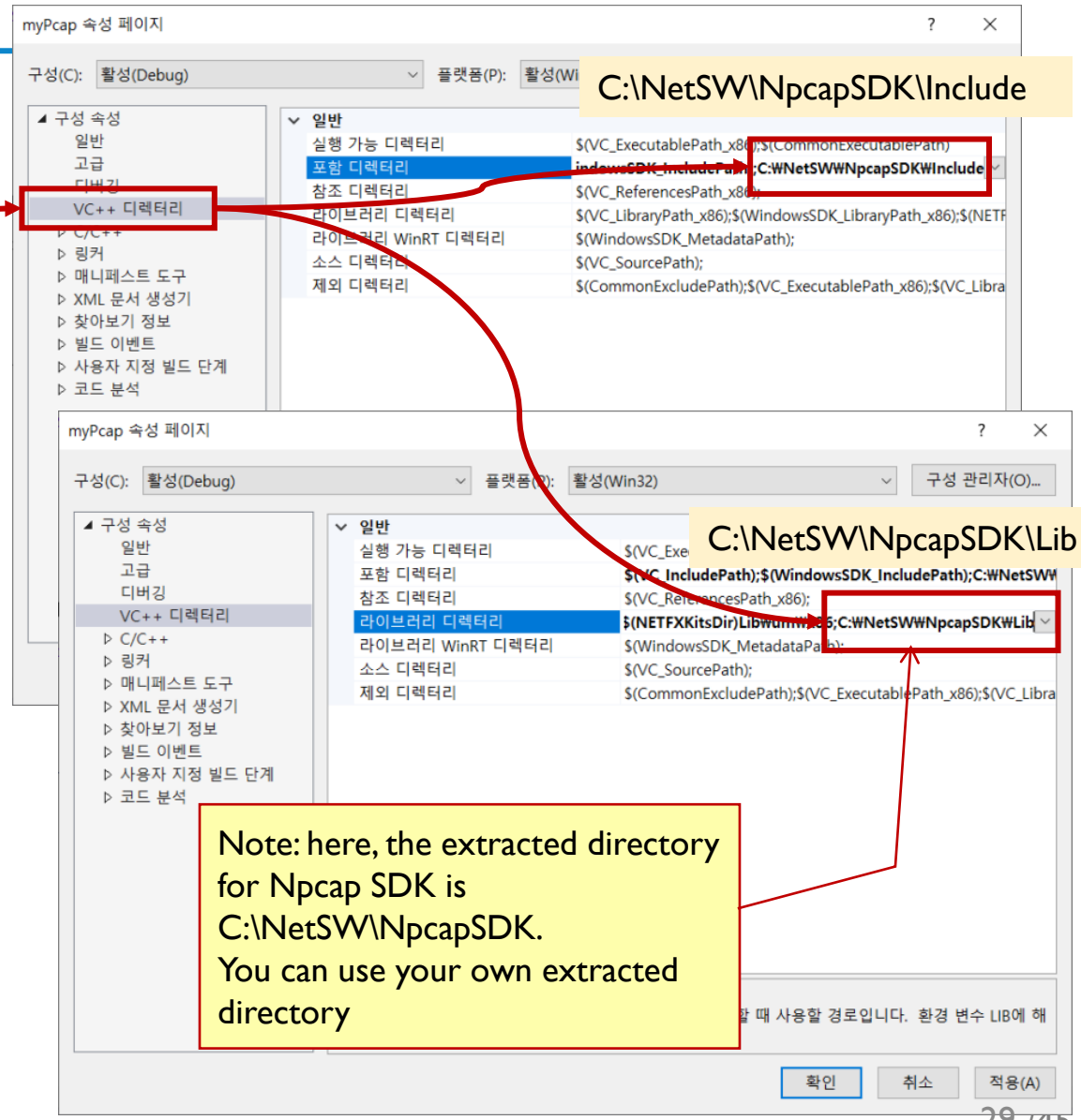
- Create a project for a Npcap programming
  - Microsoft Visual Studio



Npcap Programming with Visual Studio

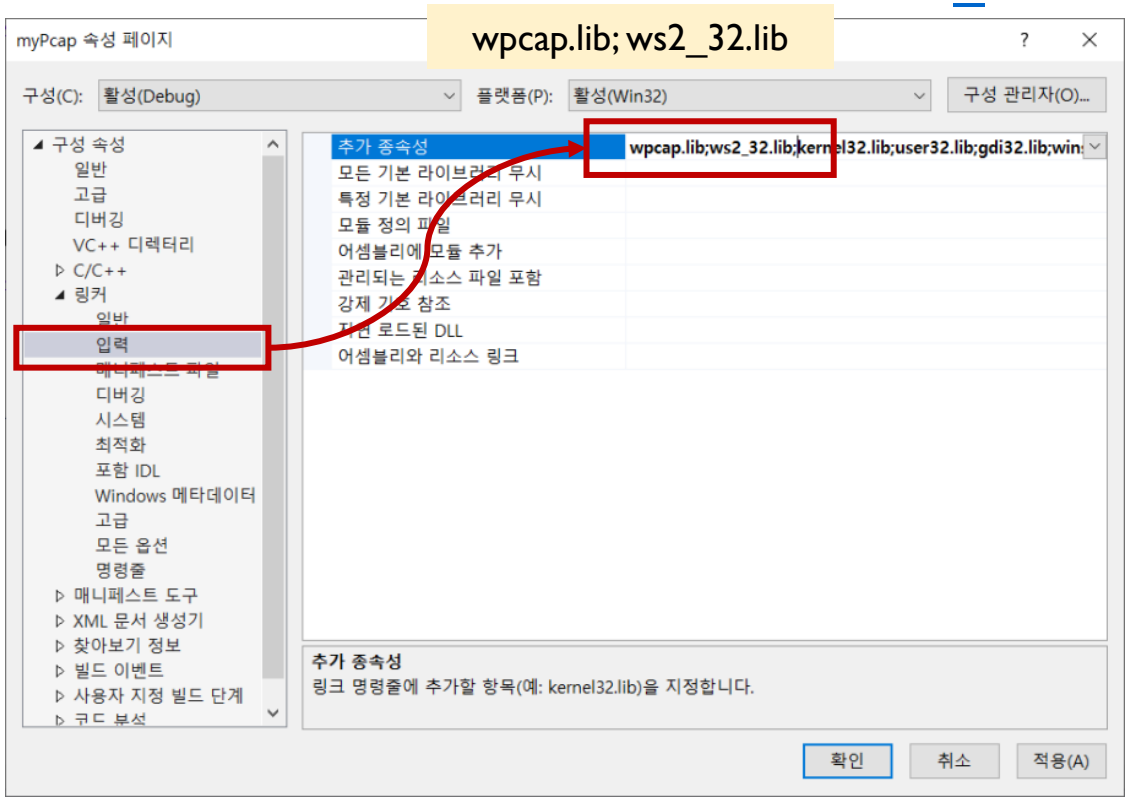
- Library and Include Files
  - add directories for include and library files

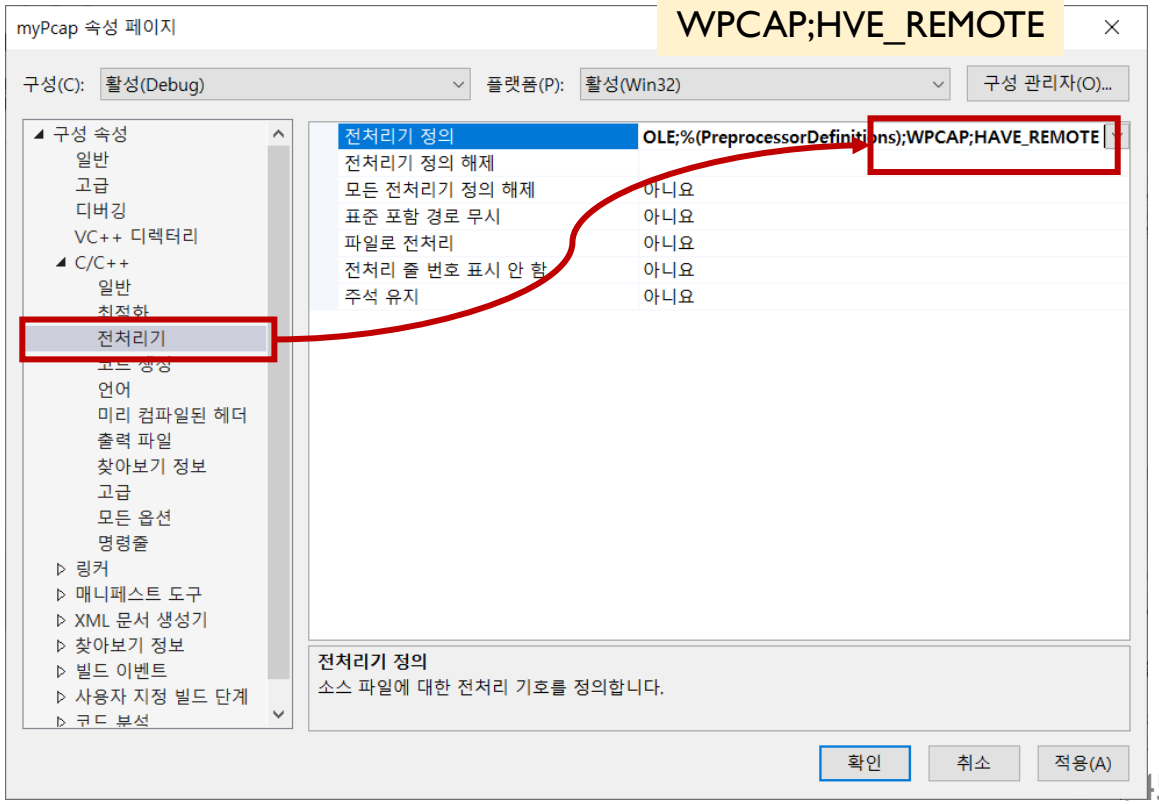




Npcap Programming with Visual Studio

- Set the linker options
  - add wpcap.lib and ws2_32.lib
  - define preprocessors
    - WPCAP for local and HAVE_REMOTE for remote captures





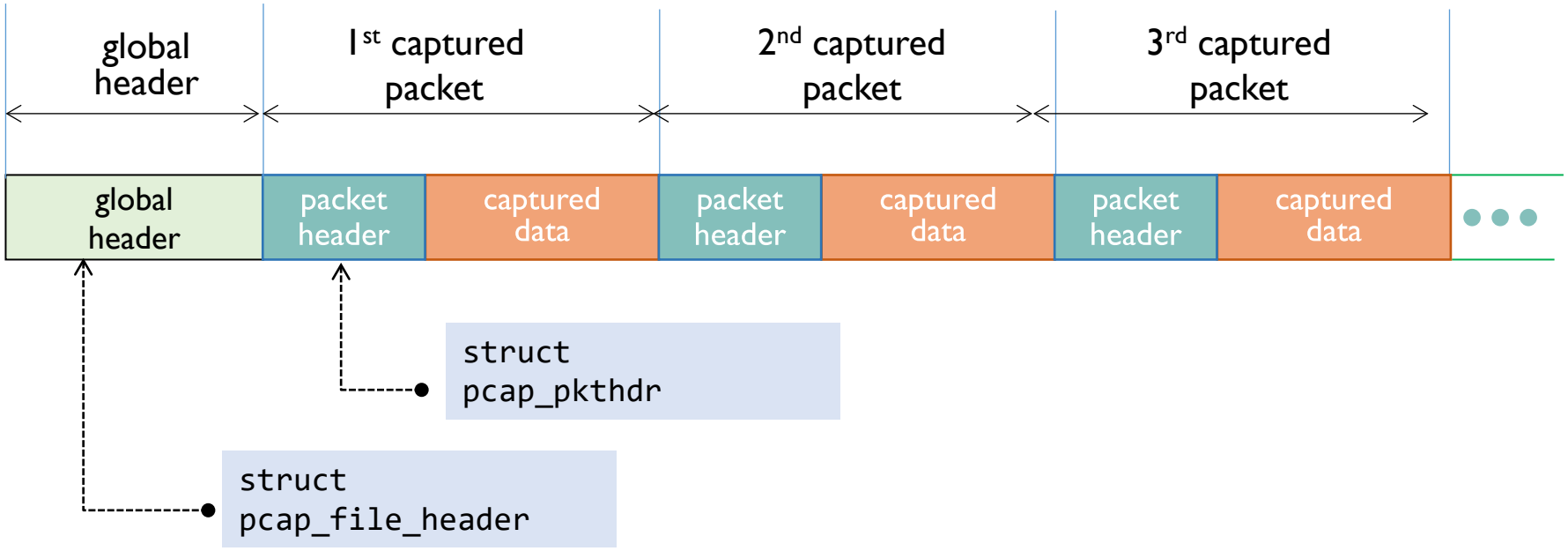
Npcap Functions

- Obtaining the list and advanced information about devices
- Capturing the packets from devices
- Capturing the packets without the callback
- Filtering the traffic
- Interpreting the packets
- Handling offline dump files
- Sending Packets
- Gathering Statistics on the network traffic

PCAP DATA STRUCTURE

Pcap Data Structure

- Data structure of captured file
  - begins with global header, and
  - repeats each captured packet with header and data



Pcap Data Structure

- Global header
  - It contains metadata about the capture
  - defined in <pcap.h>

```
struct pcap_file_header {
    bpf_u_int32         magic;          // 0xA1B2C3D4 for a valid pcap file
    u_short             version_major;
    u_short             version_minor;
    bpf_int32           thiszone;       // gmt to local correction – not used
    bpf_u_int32         sigfigs;        // accuracy of timestamps – not used
    bpf_u_int32         snaplen;        // max length saved portion of each pkt
    bpf_u_int32         linktype;       // data link type (e.g. Ehternet, 802.11, ..)
};
```

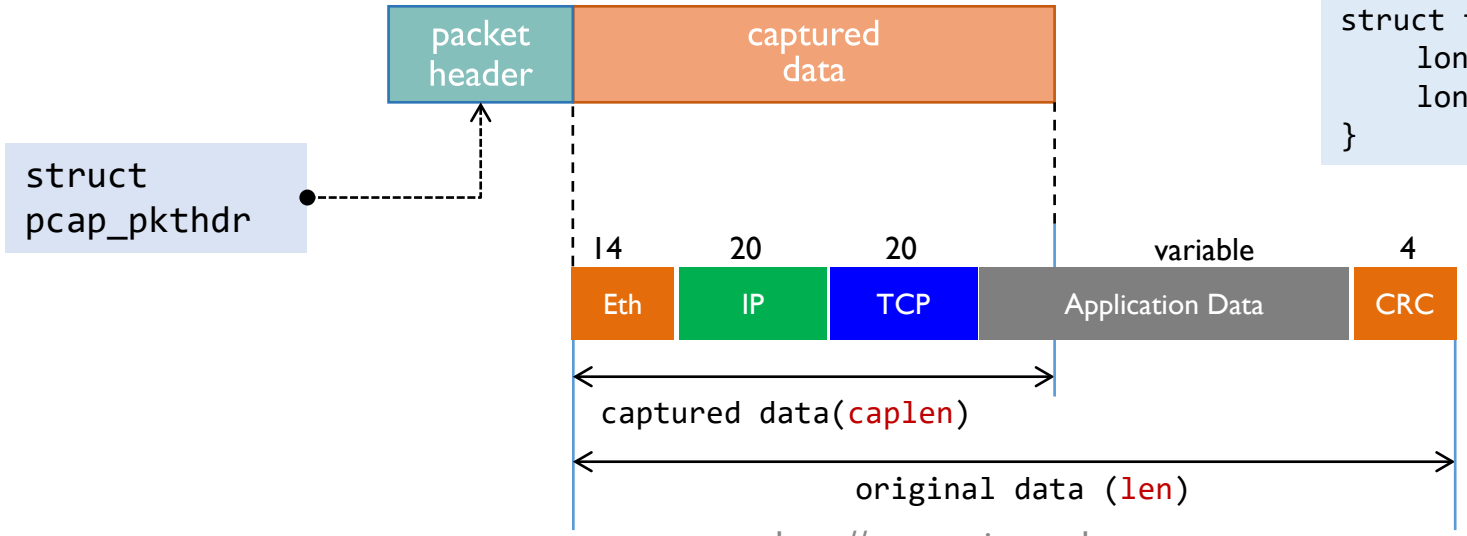
  - macros

```
#define u_int32          bpf_u_int32
#define int32            bpf_int32
```

Pcap Data Structure

- Packet header
  - it contains the time and lengths information of each captured packet

```
struct pcap_pkthdr {
    struct timeval      ts;     // time stamp (seconds and microseconds)
    bpf_u_int32         caplen; // length of captured packet (bytes)
    bpf_u_int32         len;    // total length of the original packet (bytes)
};
```




```
struct timeval {
    long   tv_sec;    // seconds
    long   tv_usec;   // microseconds
}
```

Wireshark Packet Capture Options

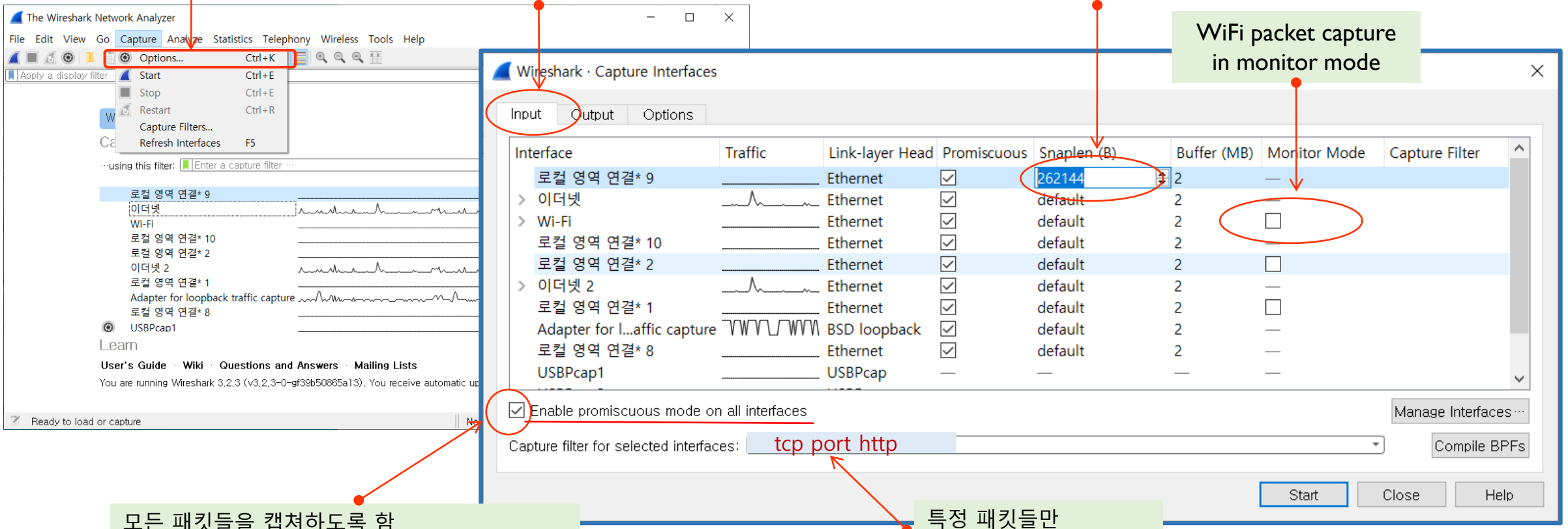
❖ Packet Capture Option - Input

capture Options

capture interface의
input 옵션 설정

caplen 지정
(1~262144)

WiFi packet capture
in monitor mode

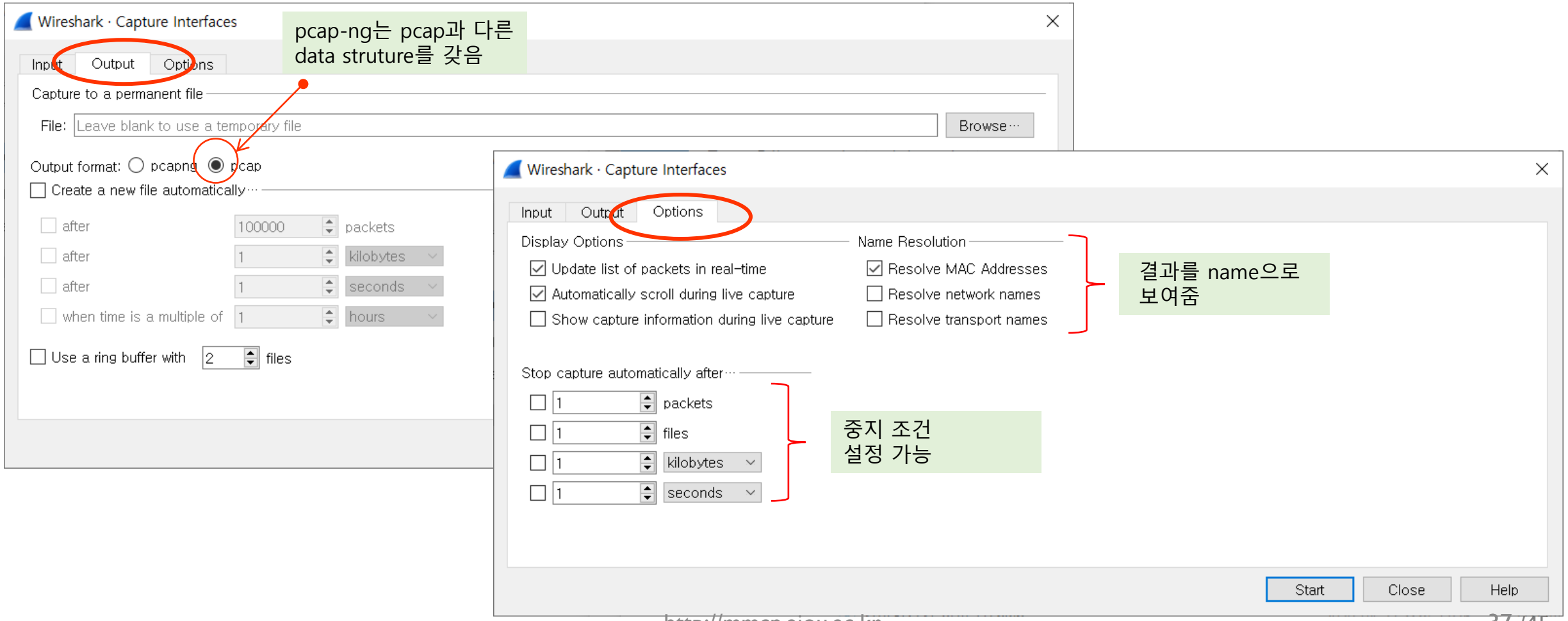


모든 패킷들을 캡쳐하도록 함
(미 설정시, 자신의 송수신 패킷들만 캡쳐됨)

특정 패킷들만
캡쳐하도록 지정함

Wireshark Packet Capture Options

❖ Packet Capture Option – Output and Options



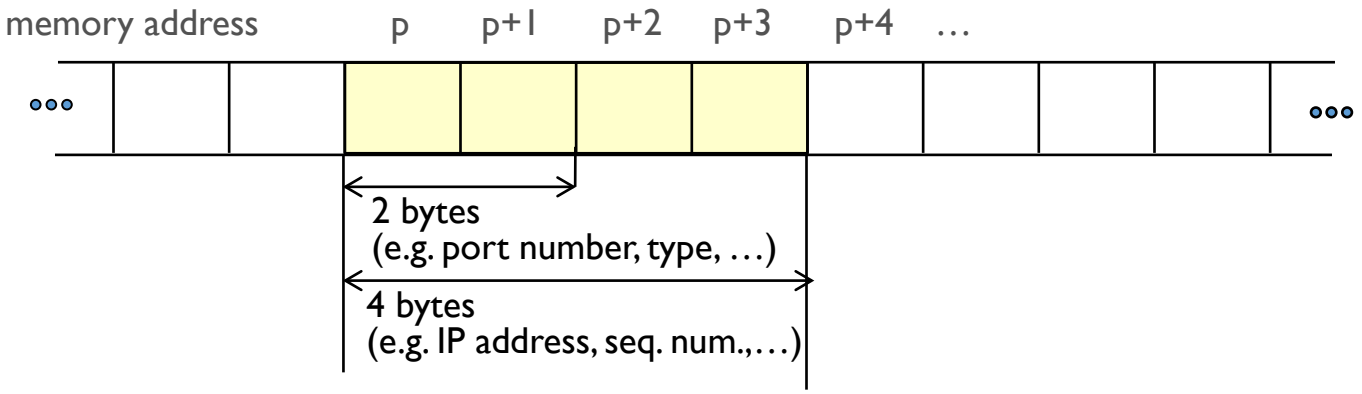
PCAP PROGRAMMING EXAMPLE – CAPTURED FILE ANALYSIS

Captured File Analysis

- Procedure
  - Step 1 : run WireShark
    - set appropriate parameters
  - Step 2 : capture packets and store them to a file
    - run some networking applications such as ftp, ping, http, …
  - Step 3 : analyze capture file using capture file analysis program
    - open the capture file
    - read each packet according to pcap data structure
    - analyze each followings:
      - Ethernet frame header
      - IP datagram header
      - TCP/UDP header
      - Application protocol header or data
    - analyze more
      - connection information
      - application usage (time, number of exchanged packets, …)

Tips on writing programs

- Reading a field with 2 or 4 bytes from a given memory address



- macro to read a value with 2 bytes

```
#define pntoh16(p)     ( (unsigned short)*((unsigned char *)(p)+0)<<8 | \
                         (unsigned short)*((unsigned char *)(p)+1)<<0 )
```

- macro to read a value with 4 bytes

```
#define pntoh32(p)     ( (unsigned short)*((unsigned char *)(p)+0)<<24 |  \
                         (unsigned short)*((unsigned char *)(p)+1)<<16 |  \
                         (unsigned short)*((unsigned char *)(p)+2)<<8  |  \
                         (unsigned short)*((unsigned char *)(p)+3)<<0  )
```

Captured File Analysis

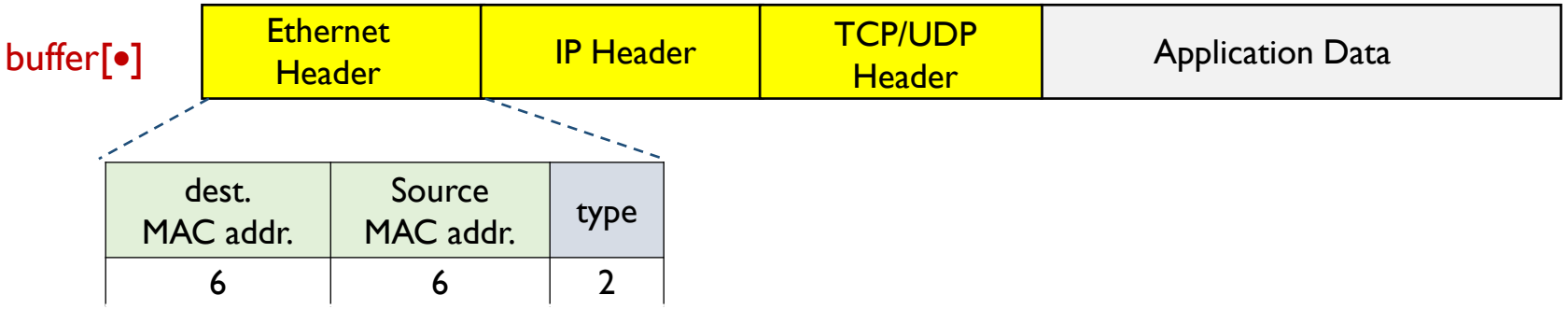
❖ Example code to get information from capture file

```
#include <pcap.h>

main( )
{
     struct pcap_file_hdr    fhdr;              // file header in captured file
     struct pcap_pkt_hdr     chdr;              // packet header
     unsigned char           buffer[2000];
     FILE                    *trace;

     trace = fopen (“my_capture_file.cap”,"rb") ;                          // open capture file

     fread ((char *)&fhdr, sizeof(fhdr), 1, trace);                        // read the global file header

     while ( fread ((char *)&chdr, sizeof(chdr), 1, trace) !=  0 )         // read packet header
     {
          fread (buffer, sizeof(unsigned char), chdr.caplen, trace);       // read packet data

          do_traffic_analysis (buffer);                  // main routing for capture packet analysis
     }
     fclose(trace);
}
```

It should be greater than caplen

Tips on writing programs

❖ Reading a field from a whole header

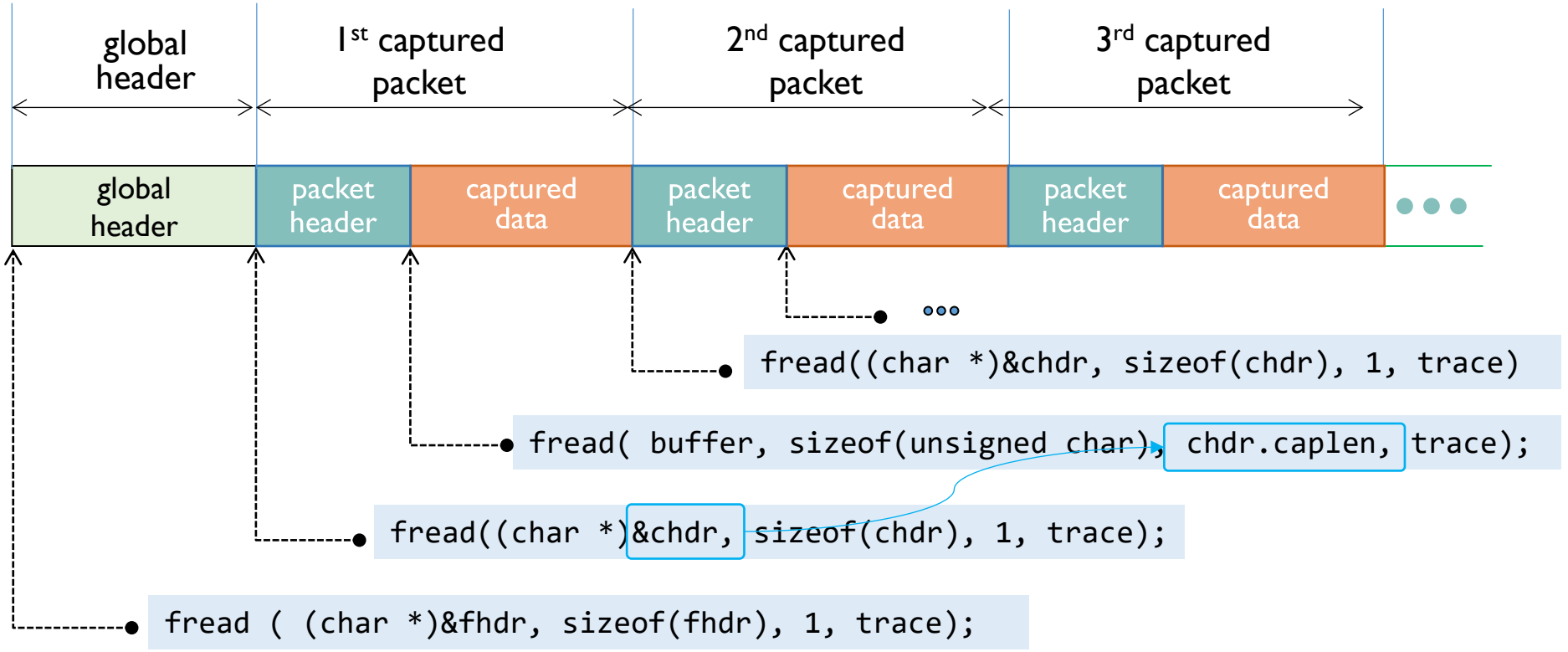



```
/* MAC address */
typedef struct mac_address {
      u_char byte1;
      u_char byte2;
      u_char byte3;
      u_char byte4;
      u_char byte5;
      u_char byte6;
} mac_address;
```

```
/* etherent header */
typedef struct eth_header{
    mac_address     smac;
    mac_address     dmac;
    u_short         type;
} eth_header;

eth_header   *eh;
eh = (eth_header *)(buffer);

if ( ntohs (eh->type) == 0x0800)
          printf("upper layer is IP\n");
```

Captured File Analysis

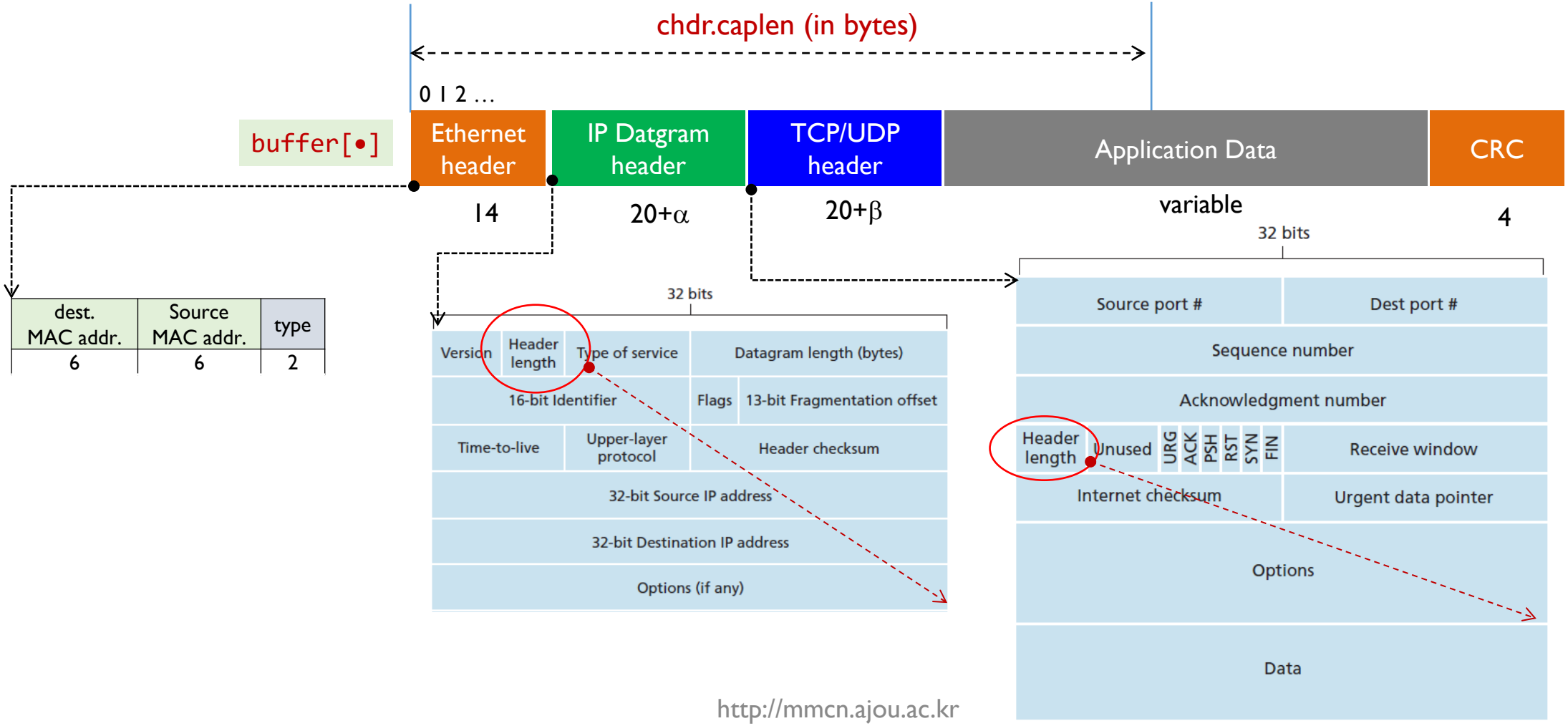
- Example – data structure




```
struct pcap_file_hdr     fhdr;
struct pcap_pkt_hdr      chdr;
```

Captured File Analysis

❖ Example – buffer (packet data)



Example – CaptureFileAnalysis.c (1/5)

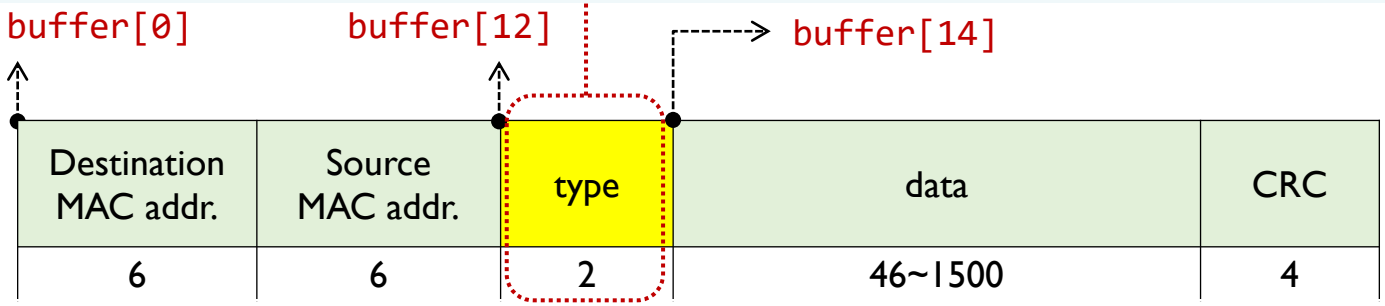
```
#include <pcap.h>
#include <stdio.h>
#include <stdlib.h>

#define LINE_LEN 16

// litereals realted to distinguishing protocols
#define ETHERTYPE_IP    0x0800
#define ETH_II_HSIZE    14      // frame size of ethernet v2
#define ETH_802_HSIZE   22      // frame size of IEEE 802.3 ethernet
#define IP_PROTO_IP     0       // IP
#define IP_PROTO_TCP    6       // TCP
#define IP_PROTO_UDP    17      // UDP
#define RTPHDR_LEN      12      // Length of basic RTP header
#define CSRCID_LEN      4       // CSRC ID length
#define EXTHDR_LEN      4       // Extension header length

unsigned long   net_ip_count;
unsigned long   net_etc_count;
unsigned long   trans_tcp_count;
unsigned long   trans_udp_count;
unsigned long   trans_etc_count;
```

Example – CaptureFileAnalysis.c (2/5)

```
void main(int argc, char **argv)
{
    struct pcap_file_header     fhdr;
    struct pcap_pkthdr          chdr;
    unsigned char               buffer[262144];
    int                         i=0;

    FILE* fin = fopen( "ccc.pcap", "rb");

    fread ( (char *)&fhdr,  sizeof(fhdr), 1, fin);

    while ( fread( (char *)&chdr,  sizeof(chdr), 1, fin) != 0 )  {  // Packet header
          fread(buffer, sizeof(unsigned char), chdr.caplen, fin);   // Packet data
          do_traffic_analysis (buffer);
          i++;
    }
    fclose(fin);

    printf("#total number of packets : %d\n",  i);
    printf("IP packets: %d\n",                 net_ip_count);
    printf("non-IP packets: %d\n",             net_etc_count);
    printf("TCP packets: %d\n",                trans_tcp_count);
}
```

Example – CaptureFileAnalysis.c (3/5)

```
void do_traffic_analysis (unsigned char *buffer)
{
    unsigned short type;

    // ethernet type check
    type = pntoh16(&buffer[12]);

    if ( type == ETHERTYPE_IP )
    {
        net_ip_count++;
        do_ip_traffic_analysis (buffer) ;
    }
    else
        net_etc_count++;
}
```



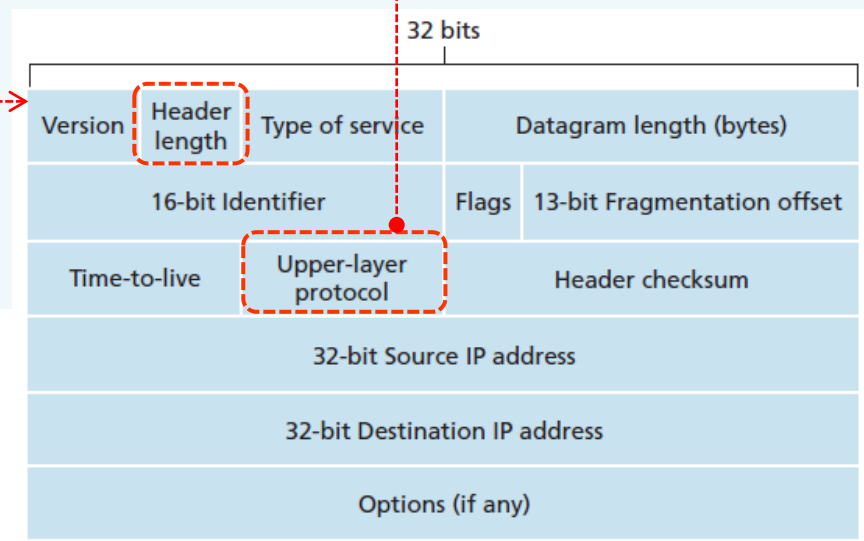
Example – CaptureFileAnalysis.c (4/5)

```
void do_ip_traffic_analysis (unsigned char *buffer)
{
    unsigned char       ip_ver, ip_hdr_len, ip_proto;
    int                 ip_offset=14;

    ip_ver      = buffer[ip_offset]>>4;          // IP version
    ip_hdr_len  = buffer[ip_offset] & 0x0f ;     // IP header length
    ip_proto    = buffer[ip_offset + 9];         // protocol above IP

    if ( ip_proto == IP_PROTO_UDP )
        trans_udp_count++;
    else if ( ip_proto == IP_PROTO_TCP )
        trans_tcp_count++;
    else
         trans_etc_count++;
}
```

buffer[0]

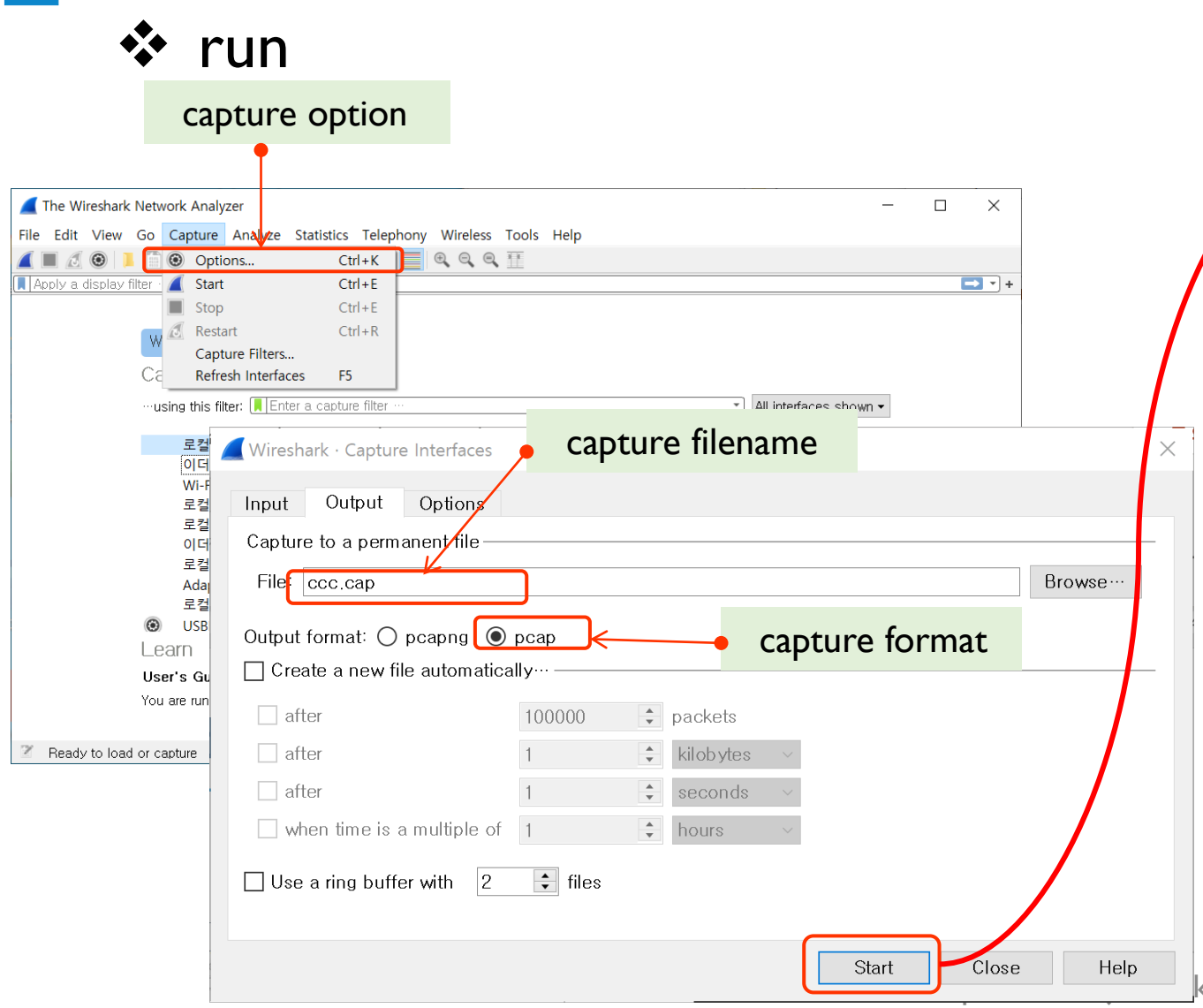
buffer[14]

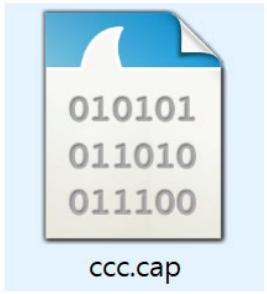
Ethernet Header	IP datagram Header	TCP/UDP Header	Application Data	CRC
14	20+α	8 for UDP 20+β for TCP	46~1500	4

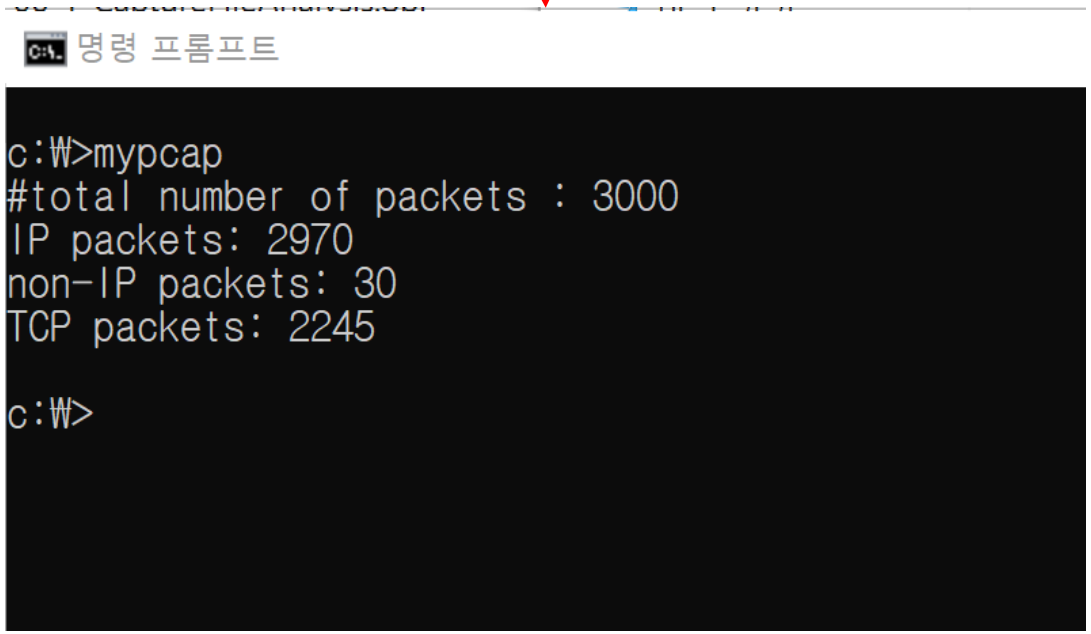


Example – CaptureFileAnalysis.c (5/5)

❖ run







PCAP PROGRAMMING EXAMPLE – CAPTURED FILE ANALYSIS2

Simple Extension of CaptureFileAnalysis.c

❖ Use of packet header structures

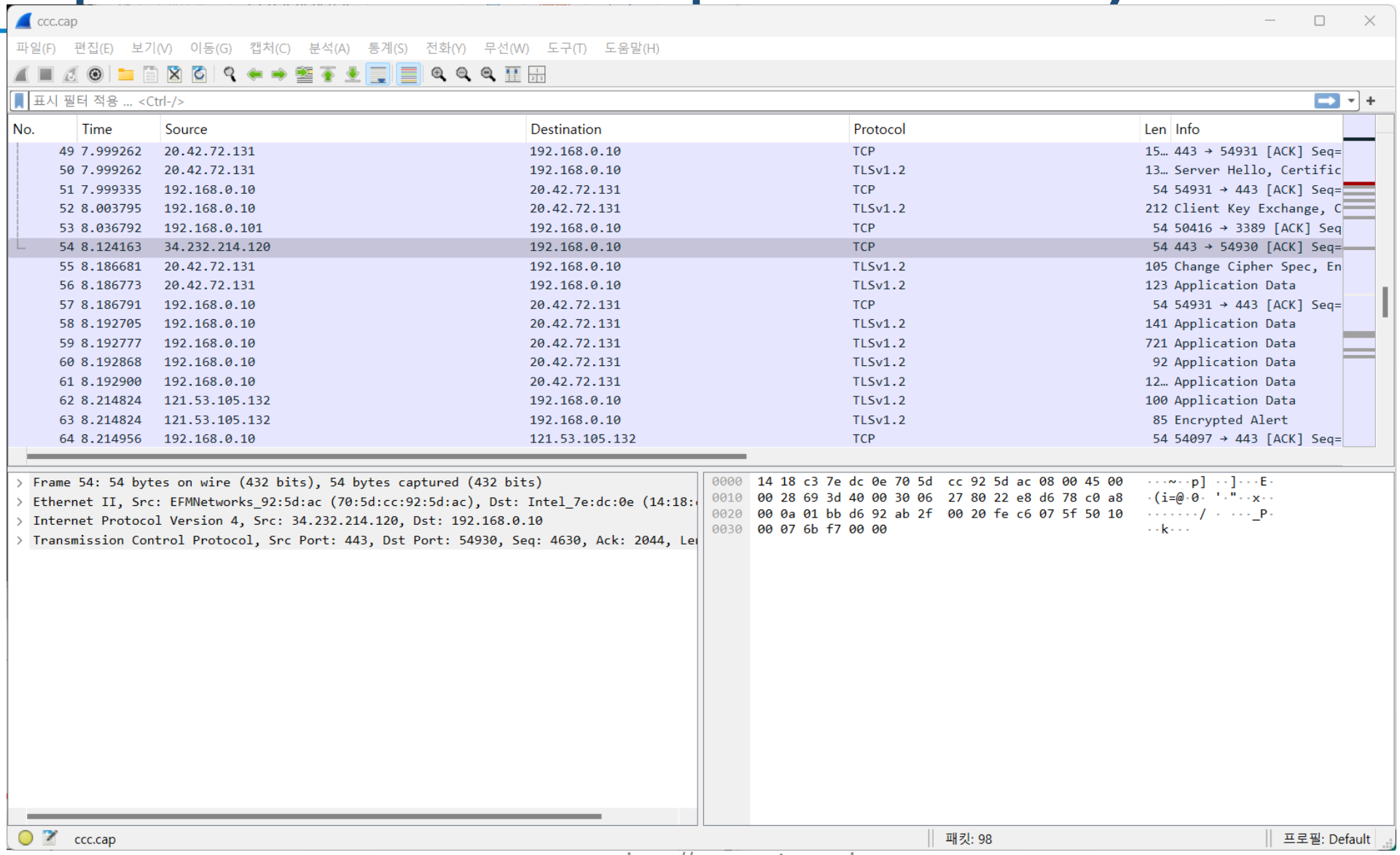
```
/* etherent header */
struct ethernet_header {
    uint8_t dest_mac[6];   // 목적지 MAC 주소
    uint8_t src_mac[6];    // 소스 MAC 주소
    uint16_t eth_type;     // Ethernet 타입
};
```

```
// IPv4 Header
struct ip_header {
    uint8_t  ihl : 4;      // IP 헤더 길이
    uint8_t  version : 4;  // IP 버전
    uint8_t  tos;          // 서비스 타입
    uint16_t tot_len;      // 전체 길이
    uint16_t id;           // 식별자
    uint16_t frag_off;     // 플래그 + 오프셋
    uint8_t  ttl;          // TTL
    uint8_t  protocol;     // 프로토콜 (TCP, UDP 등)
    uint16_t check;        // 체크섬
    uint32_t saddr;        // 소스 IP 주소
    uint32_t daddr;        // 목적지 IP 주소
};
```

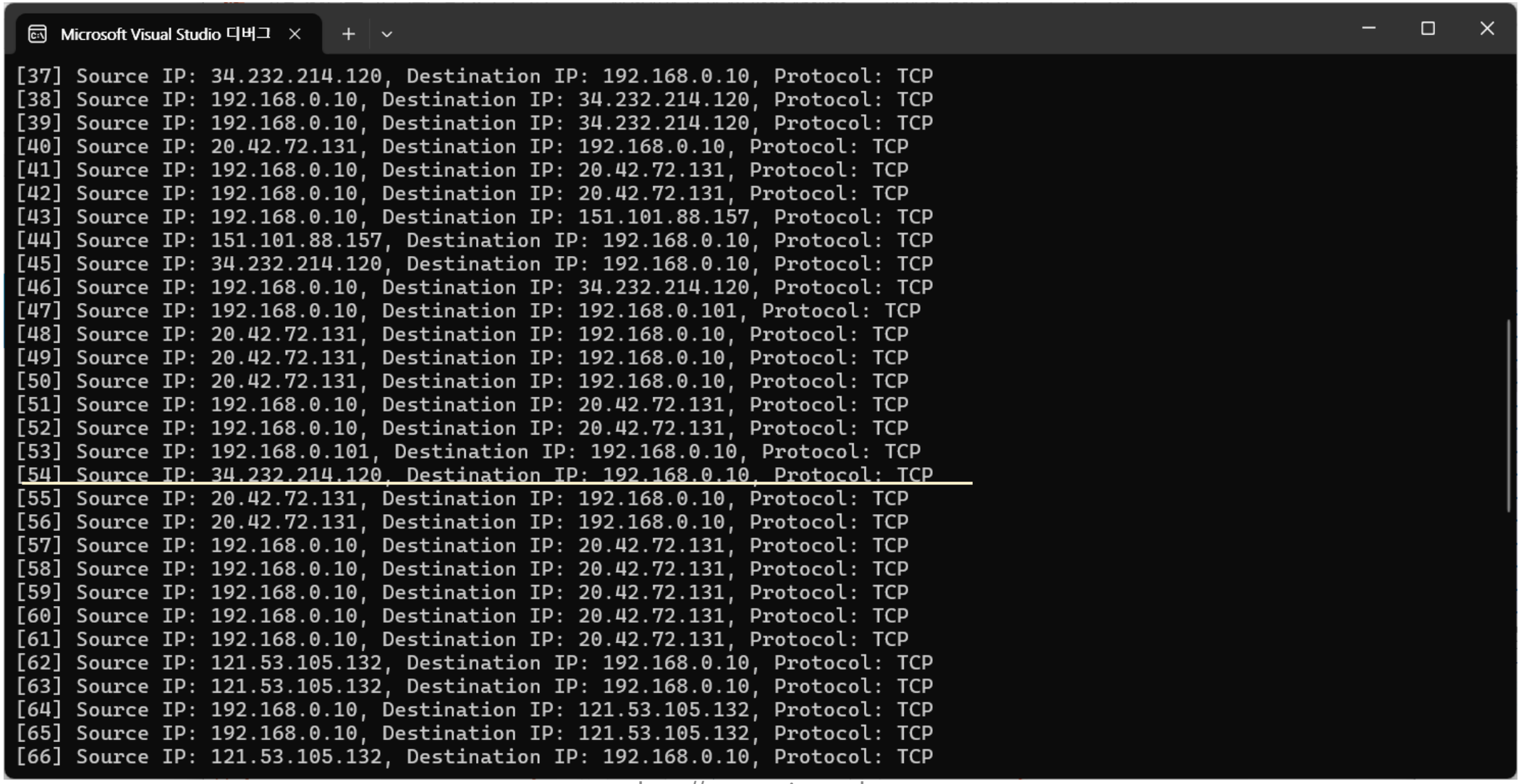
Simple Extension of CaptureFileAnalysis.c

```
...
fread(&file_header, sizeof(struct pcap_file_header), 1, file);

while (fread(&packet_header, sizeof(struct pcap_pkthdr), 1, file) == 1) {

    fread(packet_data, packet_header.caplen, 1, file); // 패킷 데이터 읽기

    struct ethernet_header *eth_hdr = (struct ethernet_header *)packet_data; // Ethernet Header

    if (ntohs(eth_hdr->eth_type) == 0x0800) {

       struct ip_header *ip_hdr = (struct ip_header *)(packet_data + 14); // IP Header

       char src_ip[INET_ADDRSTRLEN], dest_ip[INET_ADDRSTRLEN];
       inet_ntop(AF_INET, &ip_hdr->saddr, src_ip, INET_ADDRSTRLEN);
       inet_ntop(AF_INET, &ip_hdr->daddr, dest_ip, INET_ADDRSTRLEN);

       const char *protocol;
       if ( ip_hdr->protocol == 6 )
               protocol = "TCP";

       printf("Captured length: %u, Actual length: %u\n", packet_header.caplen, packet_header.len);
       ...
```

Simple Extension of CaptureFileAnalysis.c



Simple Extension of CaptureFileAnalysis.c



Questions ?

MR-IoT/AI Convergence

Future Internet & 5G/6G

Intelligent Networking with AI

MMcN

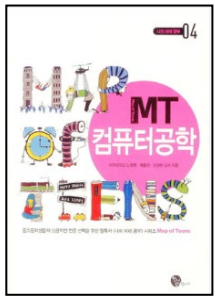
Mobile **Multimedia Convergence** Network Lab.

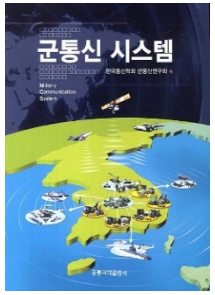
Homepage: http://mmcn.ajou.ac.kr

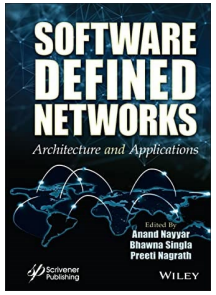
facebook: http://www.facebook.com/#!/groups/190702010947314

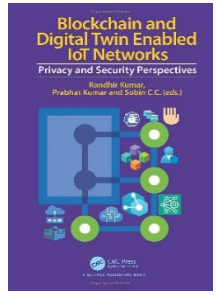
BK21 FOUR

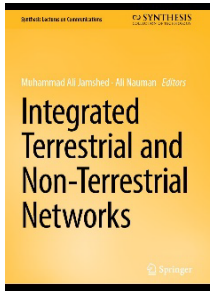
AI융합네트워크학과

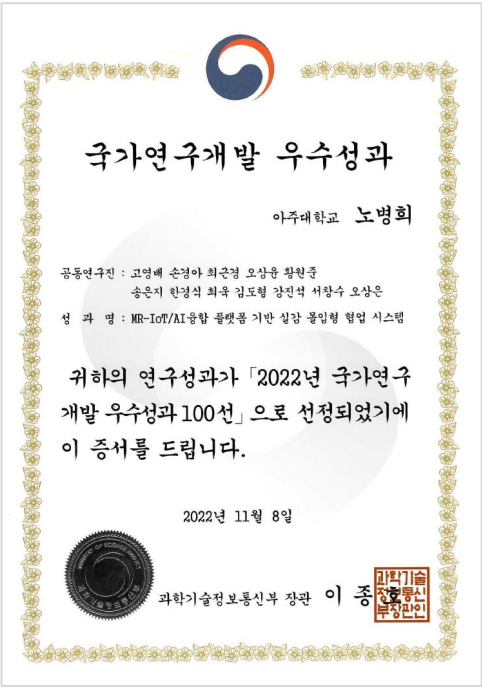












국가연구개발 우수성과

아주대학교 노병희

공동연구진 : 고영배 손경아 최준혁 오상윤 황원준
송은지 한경식 최욱 김도형 강진석 서창수 오상은

성 과 명 : MR-IoT/AI융합 플랫폼 기반 실감 몰입형 협업 시스템

귀하의 연구성과가 「2022년 국가연구개발 우수성과100선」으로 선정되었기에 이 증서를 드립니다.

2022년 11월 8일

과학기술정보통신부 장관 이 종 호